Día 5 - Listas

Hay cuatro tipos de datos de recolección en Python:

Tipo	Ordenado	Mutable	Duplicados	Indexado
Lista	Sí	Sí	Sí	Sí
Tupla	Sí	No	Sí	Sí
Set	No	No*	No	No
Diccionario	No	Sí	No (claves)	Sí

Una lista es la recopilación de diferentes tipos de datos que se ordena y modifica (mutable). Una lista puede estar vacía o puede tener diferentes elementos de tipo de datos.

Cómo crear una lista

Hay dos formas equivalentes de crear una lista vacía. Internamente ambas producen el mismo objeto `PyListObject` en el heap.

 # syntax

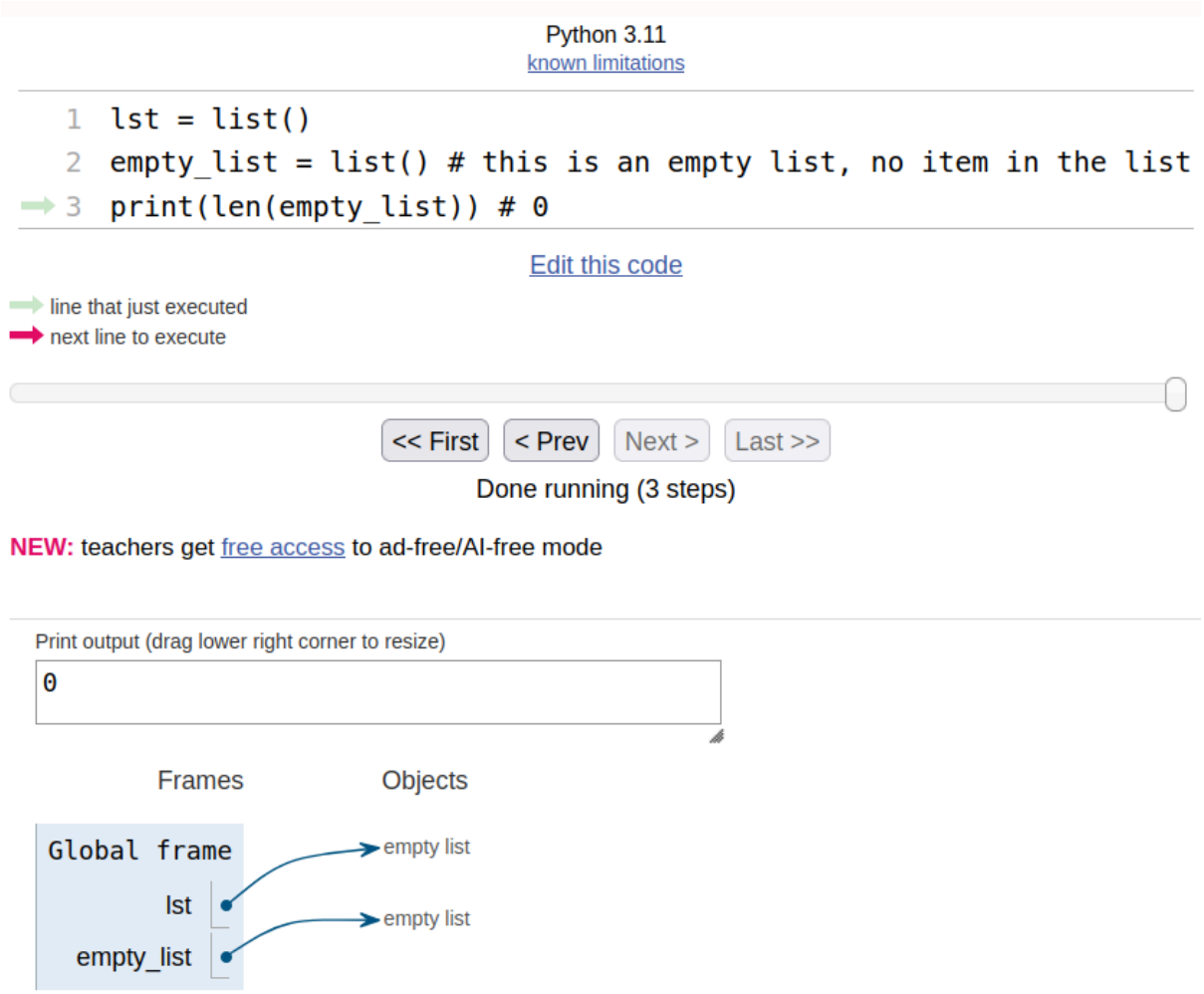
```
lst = list() # usando el constructor builtin
lst = [] # usando literal de corchetes (más común y eficiente)
```

Ejemplo1

```
lst = list() # llama al constructor list() en builtins → crea PyListObject vacío en heap
empty_list = list() # otra lista vacía independiente en el heap
print(len(empty_list)) # len() lee el campo ob_size del PyListObject → 0
```

Ejemplo2

```
empty_list = [] # this is an empty list, no item in the list
print(len(empty_list)) # 0
```



PythonTutor — Creación de listas vacías

El diagrama final muestra el Global Frame con dos entradas: `lst` y `empty_list`, ambas apuntando a recuadros etiquetados "empty list" en Objects.

Punto clave: son dos objetos distintos en el heap aunque ambos estén vacíos — cada variable guarda un puntero a su propio `PyListObject`.

Omisión: PythonTutor no muestra que `PyListObject` reserva internamente un array de punteros (`ob_item`) con capacidad inicial 0, ni el campo `ob_size=0` ni `ob_alloc` (capacidad reservada)

Modelo de memoria de listas — Diagrama completo

El diagrama incluido en el documento muestra la arquitectura real de CPython para una lista desempaquetada:

```
STACK (Variables)          HEAP: List Objects        HEAP: String Objects
─────────────────          ──────────────────        ────────────────────

lst      0xAF01  ──────►   PyListObject: lst         0x101 ► "item1"
                           [ptr0 ──────────────────────────►]
first_item 0xAF03          [ptr1 ──────────────────► 0x102 ► "item2"]
second_item 0xAF07         [ptr2 ──────────────────► 0x103 ► "item3"]
third_item 0xAF11          [ptr3 ──────────────────► 0x104 ► "item4"]
                           [ptr4 ──────────────────► 0x105 ► "item5"]
rest     0xAF01  ──────►   PyListObject: rest
                           [ptr0 ──────────────────► 0x104 ► "item4"]
                           [ptr1 ──────────────────► 0x105 ► "item5"]
```

Lo que el diagrama revela:

`lst` y `rest` son dos `PyListObject` distintos pero sus punteros internos apuntan a los mismos objetos string. `first_item`, `second_item`, `third_item` son variables en el stack que también apuntan directamente a los mismos objetos string que `lst[0]`, `lst[1]`, `lst[2]`. No hay copia de datos — todo son referencias.

Listas con valores iniciales. Utilizamos *len()* para encontrar la longitud de una lista.

Sintaxis:

python

lst = [elemento0, elemento1, elemento2]

```
fruits = ['banana', 'orange', 'mango', 'lemon']          # lista de 4 strings
vegetables = ['Tomato', 'Potato', 'Cabbage', 'Onion', 'Carrot']  # lista de 5 strings
animal_products = ['milk', 'meat', 'butter', 'yoghurt']    # lista de 4 strings
web_techs = ['HTML', 'CSS', 'JS', 'React', 'Redux', 'Node', 'MongDB']  # lista de 7 strings
countries = ['Finland', 'Estonia', 'Denmark', 'Sweden', 'Norway']      # lista de 5 strings

# imprimimos cada lista y su longitud
print('Fruits:', fruits)                          # imprime la lista completa
print('Number of fruits:', len(fruits))                  # len() → 4
print('Vegetables:', vegetables)
print('Number of vegetables:', len(vegetables))           # len() → 5
print('Animal products:', animal_products)
print('Number of animal products:', len(animal_products))   # len() → 4
```

```python
print('Web technologies:', web_techs)
print('Number of web technologies:', len(web_techs))       # len() → 7
print('Countries:', countries)
print('Number of countries:', len(countries))              # len() → 5
```

Output:

```
Fruits: ['banana', 'orange', 'mango', 'lemon']
Number of fruits: 4
Vegetables: ['Tomato', 'Potato', 'Cabbage', 'Onion', 'Carrot']
Number of vegetables: 5
Animal products: ['milk', 'meat', 'butter', 'yoghurt']
Number of animal products: 4
Web technologies: ['HTML', 'CSS', 'JS', 'React', 'Redux', 'Node', 'MongDB']
Number of web technologies: 7
Countries: ['Finland', 'Estonia', 'Denmark', 'Sweden', 'Norway']
Number of countries: 5
```

```
1
2 fruits = ['banana', 'orange', 'mango', 'lemon']                    # list
3 vegetables = ['Tomato', 'Potato', 'Cabbage','Onion', 'Carrot']      # list
4 animal_products = ['milk', 'meat', 'butter', 'yoghurt']            # list
5 web_techs = ['HTML', 'CSS', 'JS', 'React','Redux', 'Node', 'MongDB'] # lis
6 countries = ['Finland', 'Estonia', 'Denmark', 'Sweden', 'Norway']
```

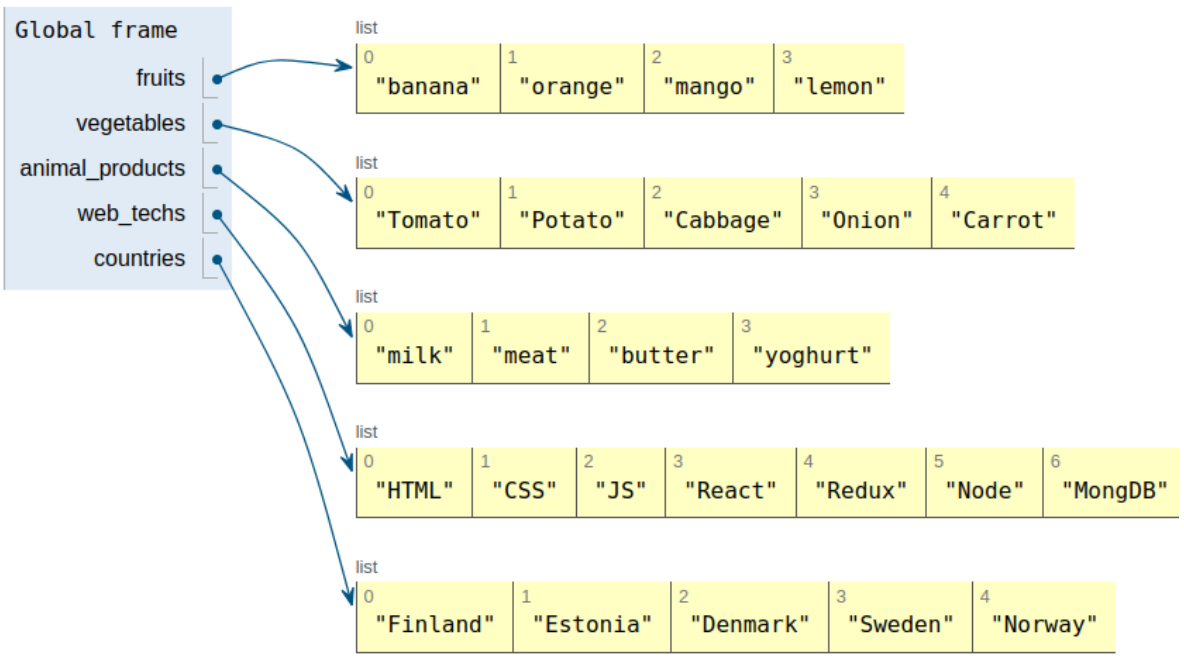
Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Done running (5 steps)

NEW: teachers get free access to ad-free/AI-free mode



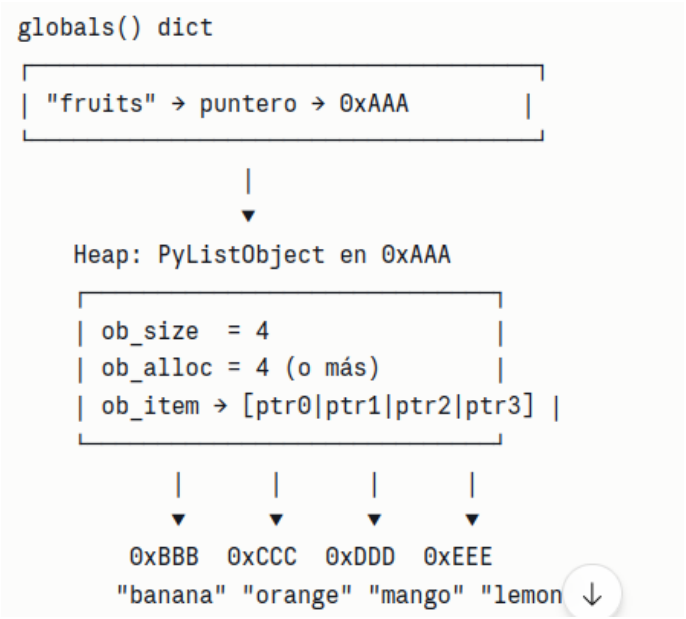
PythonTutor — Listas con valores iniciales

El diagrama muestra el Global Frame con 5 variables (`fruits`, `vegetables`, `animal_products`, `web_techs`, `countries`), cada una apuntando a un objeto lista independiente en Objects. Cada lista se visualiza como una fila de celdas indexadas desde 0.

Lo que PythonTutor muestra correctamente: cada variable es un puntero separado a su propio objeto lista.

Omisiones críticas: cada celda de la lista no contiene el string directamente sino un puntero `PyObject*` a un `PyUnicodeObject` en el heap. El `PyListObject` internamente

es un array de punteros (`ob_item[]`), no un array de valores. PythonTutor aplana esta indirección mostrando el valor string directamente en la celda.



Las listas pueden tener elementos de diferentes tipos de datos

lst = ['Asabeneh', 250, True, {'country': 'Finland', 'city': 'Helsinki'}] # cada elemento es un PyObject* distinto: str, int, bool, dict

---

Acceso a los elementos de la lista mediante indexación positiva

Cada elemento de una lista se accede mediante su índice entero. El primer índice es 0, el último es `len(lista) - 1`. Internamente el acceso es `ob_item[i]` — lectura directa de un puntero en el array, operación O(1).

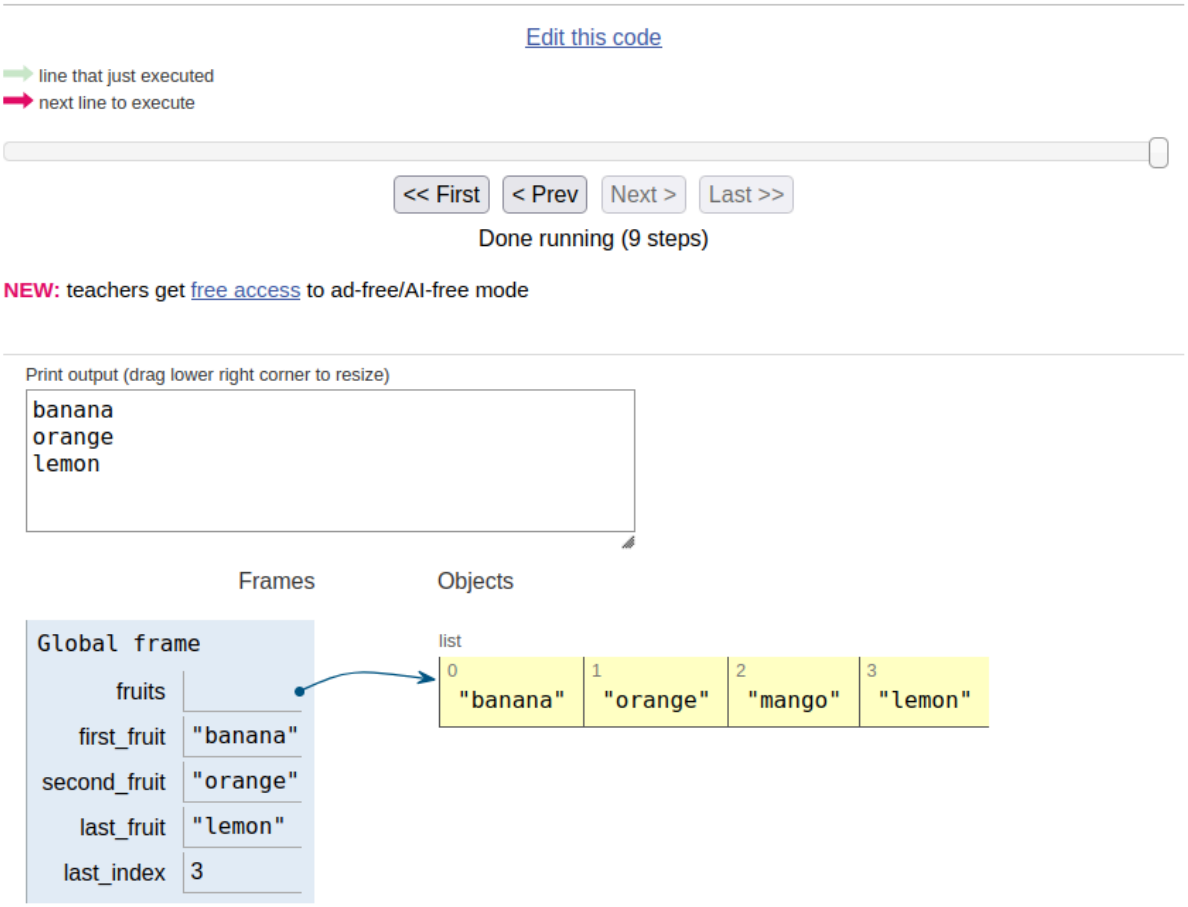
```
['banana', 'orange', 'mango', 'lemon']
     0          1          2         3
```

Sintaxis:

python

```
lista[indice]            # indice desde 0 hasta len(lista)-1

fruits = ['banana', 'orange', 'mango', 'lemon']

first_fruit = fruits[0] # ob_item[0] → puntero a "banana"
print(first_fruit) # banana

second_fruit = fruits[1] # ob_item[1] → puntero a "orange"
print(second_fruit) # orange
```

```
last_fruit = fruits[3] # ob_item[3] → puntero a "lemon"

print(last_fruit) # lemon # Calcular el último índice dinámicamente
last_index = len(fruits) - 1 # len() → 4, resta 1 → 3
last_fruit = fruits[last_index] # ob_item[3] → "lemon"
```



PythonTutor — Indexación positiva

El diagrama final muestra en el Global Frame las variables `fruits`, `first_fruit`, `second_fruit`, `last_fruit` y `last_index`. `first_fruit` y `second_fruit` aparecen con sus valores string directamente en el frame (`"banana"`, `"orange"`), mientras `fruits` apunta a la lista en Objects.

Omisión importante: `first_fruit = fruits[0]` no copia el string. Copia el puntero. Tanto `fruits[0]` como `first_fruit` apuntan al mismo `PyUnicodeObject "banana"` en el heap. PythonTutor lo muestra como si fueran valores independientes, ocultando que comparten el mismo objeto. El `ob_refcnt` de `"banana"` pasó de 1 a 2.

Acceso a los elementos de la lista mediante indexación negativa

Los índices negativos permiten acceder desde el final sin calcular la longitud. Python los resuelve internamente como `ob_item[ob_size + indice_negativo]`.

Sintaxis:

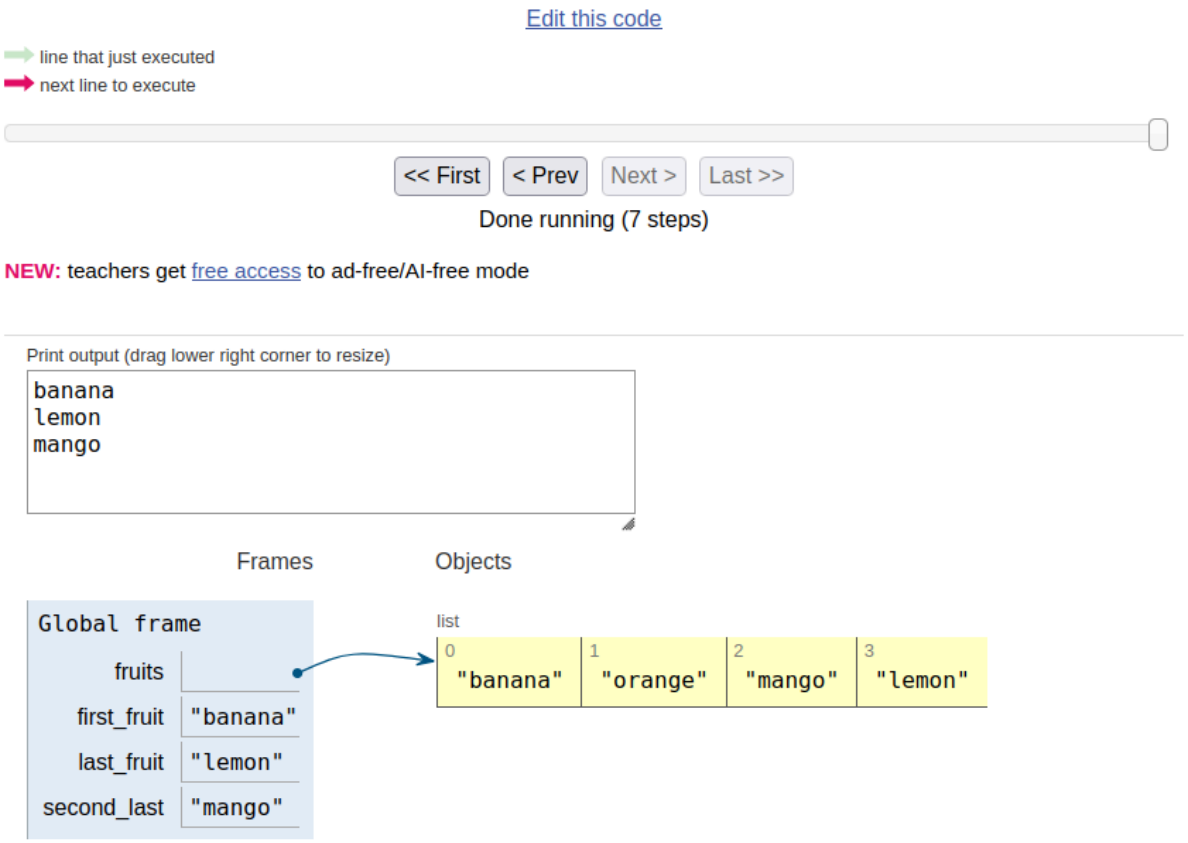
python

```
lista[-1]      # último elemento

lista[-N]      # N-ésimo elemento desde el final
```

```
['banana', 'orange', 'mango', 'lemon']
    -4          -3          -2          -1
```

python

```
fruits = ['banana', 'orange', 'mango', 'lemon']

first_fruit = fruits[-4]    # -4 → 4+(-4)=0 → ob_item[0] → "banana"
last_fruit = fruits[-1]     # -1 → 4+(-1)=3 → ob_item[3] → "lemon"
second_last = fruits[-2]    # -2 → 4+(-2)=2 → ob_item[2] → "mango"

print(first_fruit)        # banana
print(last_fruit)         # lemon
print(second_last)        # mango
```



Desembalaje de artículos de lista

El desempaquetado asigna cada elemento de la lista a una variable en una sola línea. El operador * captura los elementos restantes en una nueva lista.

Sintaxis:

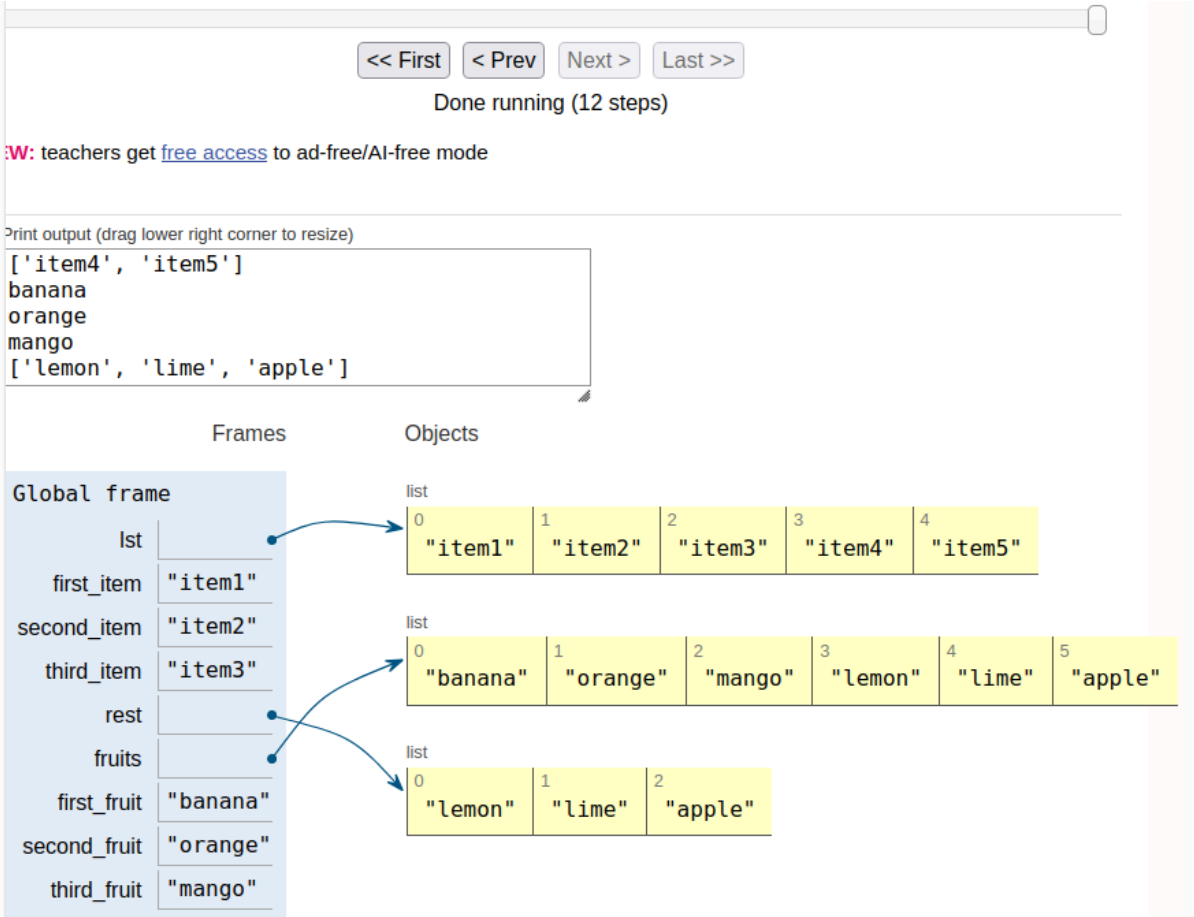
python
```
a, b, c, *resto = lista          # resto captura todo lo que no fue asignado
a, *medio, ultimo = lista        # *medio captura los elementos centrales
```

python
```
lst = ['item1', 'item2', 'item3', 'item4', 'item5']

first_item, second_item, third_item, *rest = lst
# first_item → puntero a "item1"
# second_item → puntero a "item2"
# third_item → puntero a "item3"
# *rest → crea NUEVA lista en heap con punteros a "item4" y "item5"

print(first_item)   # item1
print(second_item)  # item2
```

```
print(third_item)   # item3
print(rest)         # ['item4', 'item5']
```

 # First Example

```
fruits = ['banana', 'orange', 'mango', 'lemon','lime','apple']
first_fruit, second_fruit, third_fruit, *rest = fruits

print(first_fruit)     # banana
print(second_fruit)    # orange
print(third_fruit)     # mango

print(rest)            # ['lemon','lime','apple']
```



PythonTutor — Desempaquetado (Ejemplo 1 con fruits)

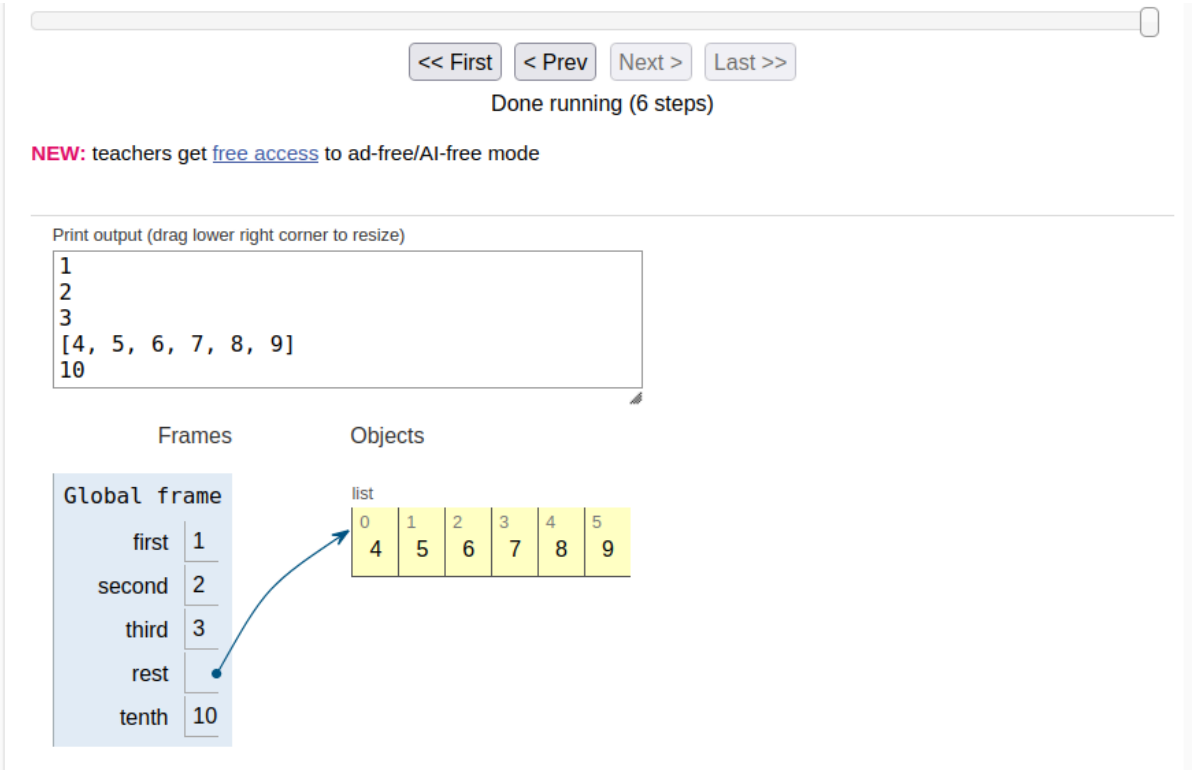
El diagrama muestra en Objects tres listas distintas: la lista original `lst` con 5 elementos, la lista `fruits` con 6 elementos, y una tercera lista más pequeña apuntada por `rest`. En el Global Frame, `first_item`, `second_item`, `third_item` contienen sus valores directamente.

Omisión crítica: `rest` no contiene copias de los strings "lemon", "lime", "apple". Es un nuevo `PyListObject` cuyo `ob_item[]` contiene punteros a los mismos objetos string que `fruits[3]`, `fruits[4]`, `fruits[5]`. Los strings no se duplican — se comparte la referencia. PythonTutor muestra los strings como si fueran independientes.

 # Second Example about unpacking list

python

```python
# Ejemplo 2 — *rest en posición central
first, second, third, *rest, tenth = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# first=1, second=2, third=3
# *rest → nueva lista [4,5,6,7,8,9]
# tenth=10 — Python asigna el último elemento a tenth antes de construir rest

print(first)   # 1
print(second)  # 2
print(third)   # 3
print(rest)    # [4, 5, 6, 7, 8, 9]
print(tenth)   # 10
```



PythonTutor — Desempaquetado Ejemplo 2

El diagrama muestra `first=1`, `second=2`, `third=3`, `tenth=10` directamente en el Global Frame (son enteros pequeños, internados por CPython). `rest` apunta a una lista en Objects con los índices 0-5 conteniendo 4,5,6,7,8,9.

Omisión: Global Frame — qué contiene realmente cada variable

El Global Frame es un `dict` donde cada clave es el nombre de la variable como string y cada valor es una referencia (puntero) a un objeto en el heap.

```
globals() dict
┌──────────────────────────────────────────┐
| "first"  → puntero → PyLongObject  (1)   |
| "second" → puntero → PyLongObject  (2)   |
| "third"  → puntero → PyLongObject  (3)   |
| "rest"   → puntero → PyListObject        |
| "tenth"  → puntero → PyLongObject  (10)  |
└──────────────────────────────────────────┘
```

El `*` en el desempaquetado NO es desreferenciación

El `*rest` en el contexto de desempaquetado **no es el operador de desreferenciación de C**. Es una sintaxis de Python que le dice al intérprete "todo lo que no fue asignado a las otras variables va a esta". El resultado es que `rest` en el namespace queda como una variable normal que apunta a un `PyListObject` nuevo en el heap.

"rest" → puntero → PyListObject en 0xAAA

Ese `PyListObject` internamente tiene su propio array `ob_item[]` con punteros a los enteros del pool.

Diagrama completo de memoria

```
globals() dict (stack conceptual)
┌──────────────────────────────┐
| "first"  ──→ 0xF01           |
| "second" ──→ 0xF02           |
| "third"  ──→ 0xF03           |
| "rest"   ──→ 0xAAA           |
| "tenth"  ──→ 0xF0A           |
└──────────────────────────────┘
     |         |         |         |              |
     ▼         ▼         ▼         ▼              ▼
  Pool de enteros internados             Heap: PyListObject
┌────────┐ ┌────────┐ ┌────────┐  0xAAA            ┌──────────────┐
|int: 1  | |int: 2  | |int: 3  |  ┌──────────────→ |ob_size = 6 |
|ob_ref=N| |ob_ref=N| |ob_ref=N|  |                |ob_item[]   |
└────────┘ └────────┘ └────────┘  |                |  [0]──────→|──→ int: 4
                                   |                |  [1]──────→|──→ int: 5
                                   |                |  [2]──────→|──→ int: 6
     ▼                             |                |  [3]──────→|──→ int: 7
┌────────┐                         |                |  [4]──────→|──→ int: 8
|int: 10 |                         |                |  [5]──────→|──→ int: 9
|ob_ref=N|                         |                └──────────────┘
└────────┘                         |
                                   |
                                   |
              "rest" ──────────────┘
```

Por qué `first`, `second`, `third`, `tenth` apuntan directamente al pool

CPython pre-crea e interna los enteros en el rango **-5 a 256** al iniciar el intérprete. Cuando Python evalúa el literal `[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]`, no crea objetos nuevos para los valores dentro de ese rango — simplemente obtiene punteros a los objetos ya existentes en el pool.

 # Lo que NO ocurre (incorrecto):

first = nuevo PyLongObject(1) ← NO, no se crea nada

 # Lo que SÍ ocurre (correcto):

first = puntero al PyLongObject(1) que ya existe en el pool

Por eso en Python Tutor `first`, `second`, `third`, `tenth` aparecen con sus valores directamente en el frame — porque son referencias a objetos inmutables pre-existentes. PythonTutor omite que todos esos enteros son el mismo objeto compartido cada vez que aparezca ese valor en el programa.

Por qué rest sí crea un objeto nuevo

`*rest` fuerza la creación de un `PyListObject` nuevo aunque todos sus elementos sean enteros internados. La lista en sí no puede ser internada — solo su contenido lo es.

ob_item[] de rest contiene punteros a objetos del pool:

rest[0] → mismo objeto int(4) del pool

rest[1] → mismo objeto int(5) del pool

...

rest[5] → mismo objeto int(9) del pool

Modificar `rest[0] = 99` no afecta el pool — solo reemplaza el puntero en `ob_item[0]` de este `PyListObject`. El objeto `int(4)` en el pool no se toca.

Orden de evaluación del desempaquetado

Python resuelve la asignación en este orden:

1. Evalúa el lado derecho → crea el PyListObject [1..10] en heap

2. Asigna variables con posición fija desde la izquierda: first=1, second=2, third=3

3. Asigna variables con posición fija desde la derecha: tenth=10

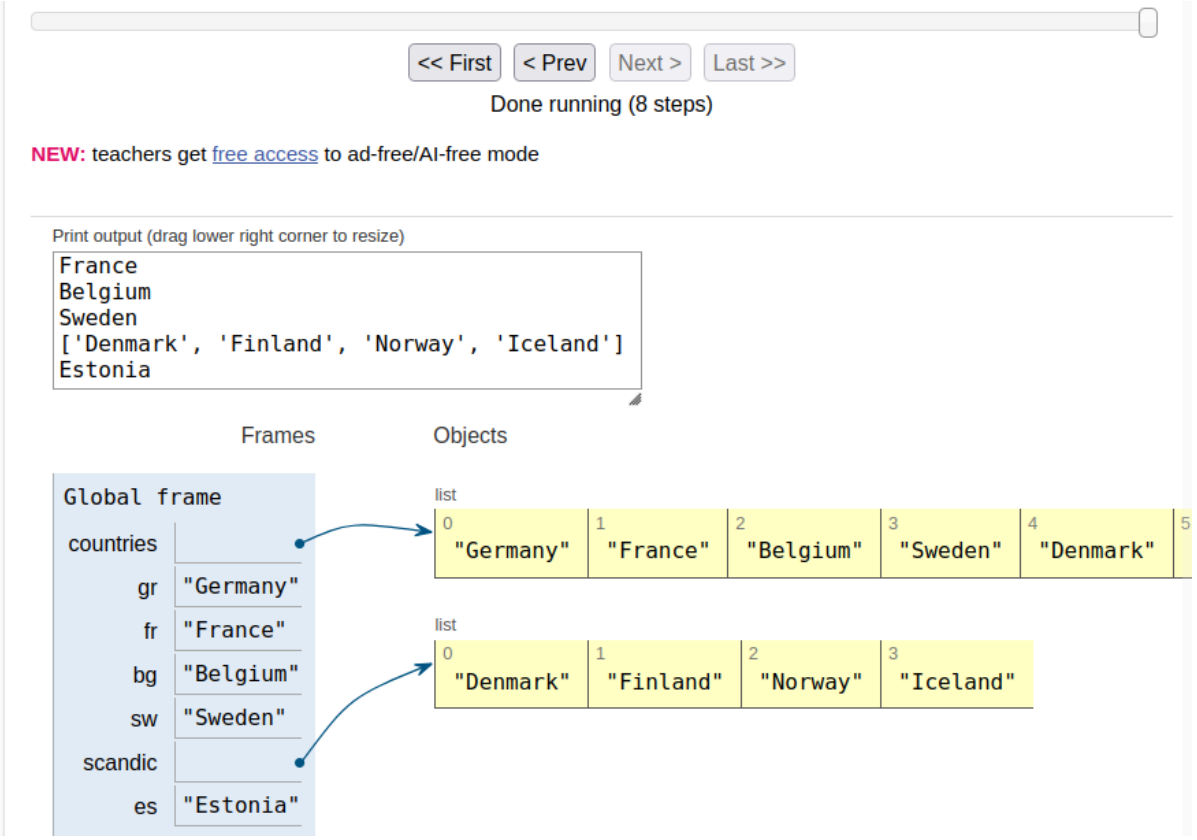
4. Los elementos restantes [4,5,6,7,8,9] → nuevo PyListObject → rest

5. El PyListObject [1..10] original decrementa ob_refcnt → si llega a 0 se libera

---

 # Third Example about unpacking list

```
# Ejemplo 3 — *rest con países

countries = ['Germany', 'France', 'Belgium', 'Sweden', 'Denmark', 'Finland', 'Norway', 'Iceland', 'Estonia']

gr, fr, bg, sw, *scandic, es = countries # gr="Germany", fr="France", bg="Belgium", sw="Sweden" # *scandic → nueva lista con ["Denmark","Finland","Norway","Iceland"] # es="Estonia" → último elemento asignado antes de construir scandic

print(gr) # Germany
print(fr) # France
print(bg) # Belgium
print(sw) # Sweden
print(scandic) # ['Denmark', 'Finland', 'Norway', 'Iceland']
print(es) # Estonia
```



PythonTutor — Desempaquetado Ejemplo 3

El diagrama muestra `countries` apuntando a la lista original de 9 elementos. `scandic` apunta a una segunda lista más pequeña con 4 elementos. Las variables escalares (`gr`, `fr`, etc.) muestran sus valores directamente en el frame.

Omisión: la lista `scandic` comparte los mismos objetos string que `countries` — no hay copia de datos.

Slicing — Rebanar elementos de una lista.

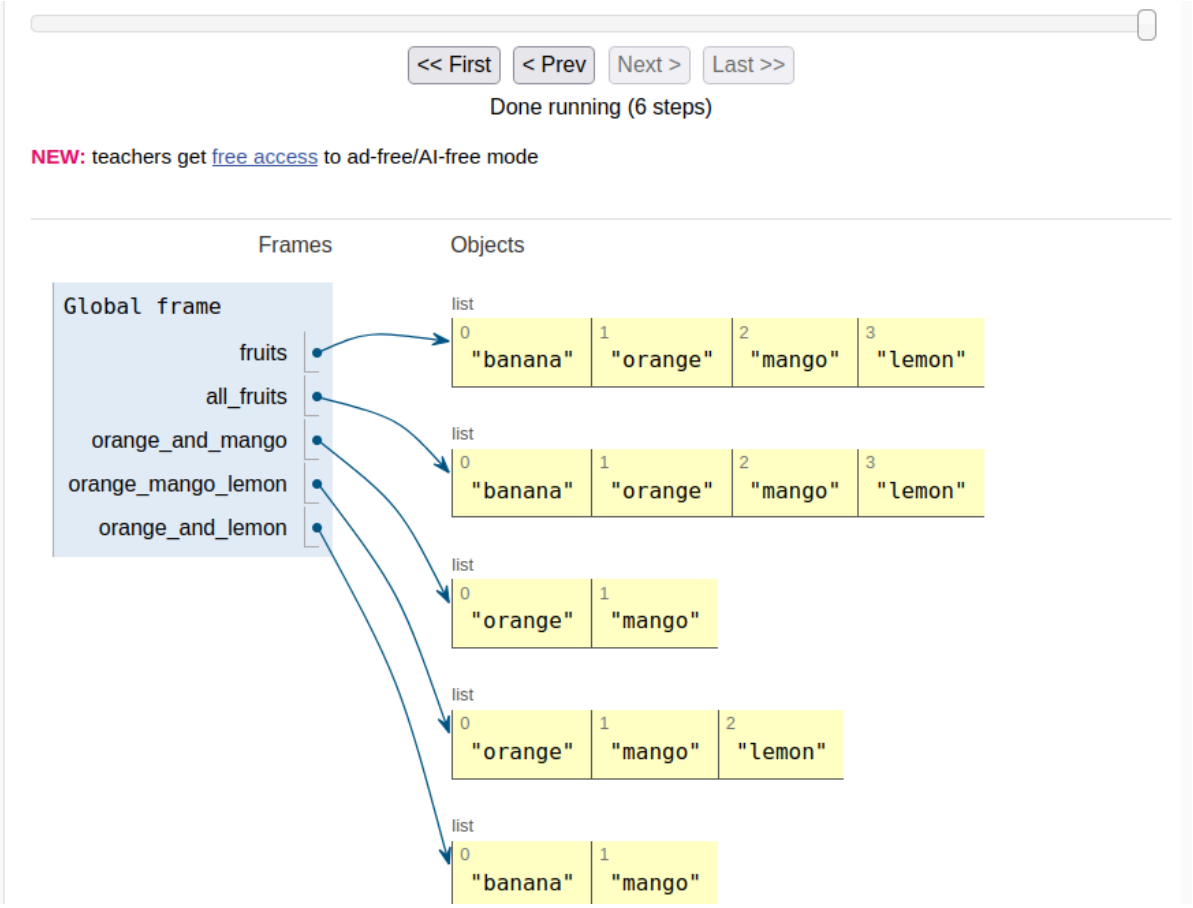
El slicing extrae una sublista. **Siempre crea un nuevo `PyListObject`** con punteros copiados del original — los strings apuntados no se duplican.

Sintaxis:

```
python
lista[start:stop]          # desde start hasta stop-1
lista[start:stop:step]     # con salto
lista[start:]              # desde start hasta el final
lista[:stop]               # desde el inicio hasta stop-1
lista[::-1]                # recorre en orden inverso
```

Indexación positiva: Podemos especificar un rango de índices positivos especificando el inicio, final y paso, el valor de retorno será una nueva lista. (valores predeterminados para inicio = 0, final = len(lst) - 1 (último elemento), paso = 1)

python
```
fruits = ['banana', 'orange', 'mango', 'lemon']

all_fruits = fruits[0:4]       # nueva lista con punteros a índices 0,1,2,3 → copia total
all_fruits = fruits[0:]        # equivalente: sin stop → hasta el final
orange_and_mango = fruits[1:3]  # índices 1,2 → ["orange","mango"] (stop=3 no incluido)
orange_mango_lemon = fruits[1:] # desde índice 1 hasta el final →
["orange","mango","lemon"]
orange_and_lemon = fruits[::2]  # step=2 → índices 0,2 → ["banana","mango"]
```



PythonTutor — Slicing positivo

El diagrama muestra `fruits` apuntando a la lista original y cuatro variables (`all_fruits`, `orange_and_mango`, `orange_mango_lemon`, `orange_and_lemon`) apuntando a listas distintas en Objects. **Omisión clave:** todas estas listas comparten los mismos objetos string — `orange_and_mango[0]` y `fruits[1]` apuntan al mismo `PyUnicodeObject "orange"`. El slicing copia punteros, no objetos.

Indexación negativa: Podemos especificar un rango de índices negativos especificando el inicio, final y paso, el valor de retorno será una nueva lista.

python
```
fruits = ['banana', 'orange', 'mango', 'lemon']

all_fruits = fruits[-4:]          # -4 → índice 0, hasta el final → lista completa
orange_and_mango = fruits[-3:-1]    # índices 1,2 → ["orange","mango"] (-1 no incluido)
orange_mango_lemon = fruits[-3:]    # desde índice 1 hasta el final →
["orange","mango","lemon"]
reverse_fruits = fruits[::-1]      # step=-1 → recorre de atrás hacia adelante → nueva lista
invertida
```

Modificar las listas.

A diferencia de los strings, las listas son **mutables**: se puede reemplazar el puntero en cualquier posición del array `ob_item[]` sin crear una lista nueva.

La lista es una colección ordenada mutable o modificable de artículos. Modifiquemos la lista de frutas.

Sintaxis:

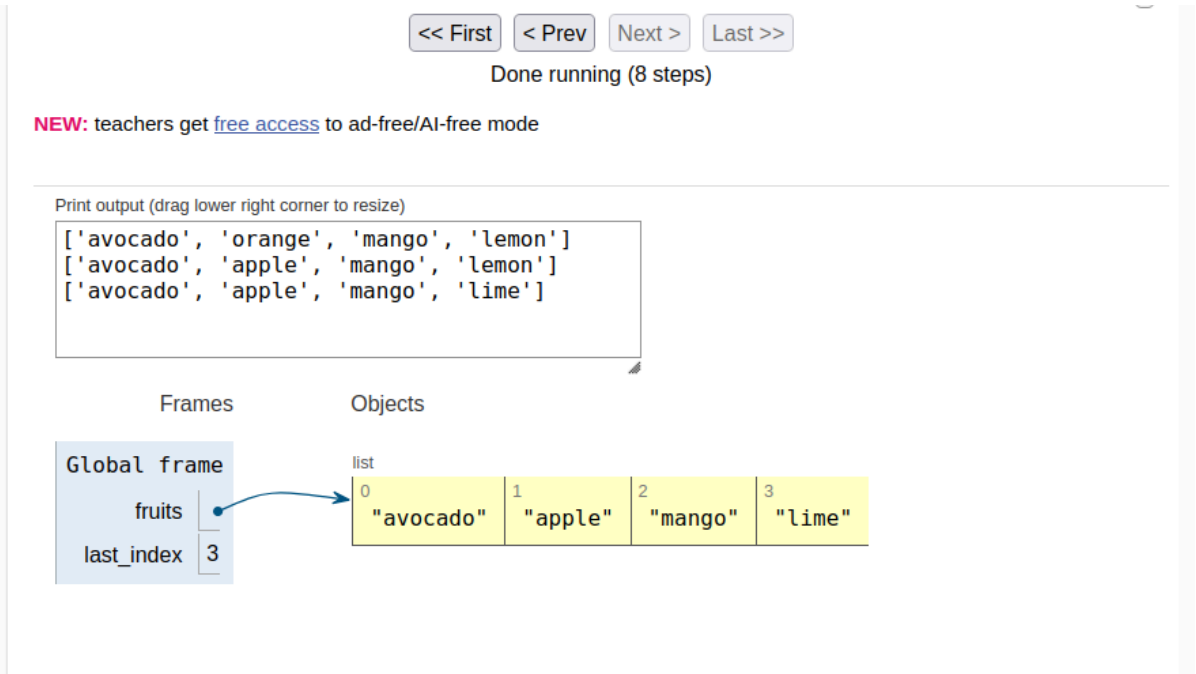
python
```
lista[indice] = nuevo_valor     # reemplaza el puntero en ob_item[indice]
```

python
```
fruits = ['banana', 'orange', 'mango', 'lemon']

fruits[0] = 'avocado'       # ob_i tem[0] ahora apunta al nuevo objeto "avocado"
                        # el ob_refcnt de "banana" decrementa
print(fruits)             # ['avocado', 'orange', 'mango', 'lemon']

fruits[1] = 'apple'        # ob_item[1] reemplaza puntero a "orange" por puntero a "apple"
print(fruits)             # ['avocado', 'apple', 'mango', 'lemon']

last_index = len(fruits) - 1    # 3
fruits[last_index] = 'lime'     # ob_item[3] reemplaza "lemon" por "lime"
print(fruits)             # ['avocado', 'apple', 'mango', 'lime']
```

PythonTutor — Modificación de lista

El diagrama final muestra la misma lista en Objects pero con los contenidos actualizados: `"avocado"`, `"apple"`, `"mango"`, `"lime"`. En el Global Frame aparece `last_index = 3`.

Omisión: los objetos string antiguos (`"banana"`, `"orange"`, `"lemon"`) no desaparecen instantáneamente — su `ob_refcnt` se decrementa y si llega a 0 son liberados. La lista en Objects es el **mismo** `PyListObject` (misma dirección), solo cambiaron los punteros internos.



Comprobar los artículos en una lista

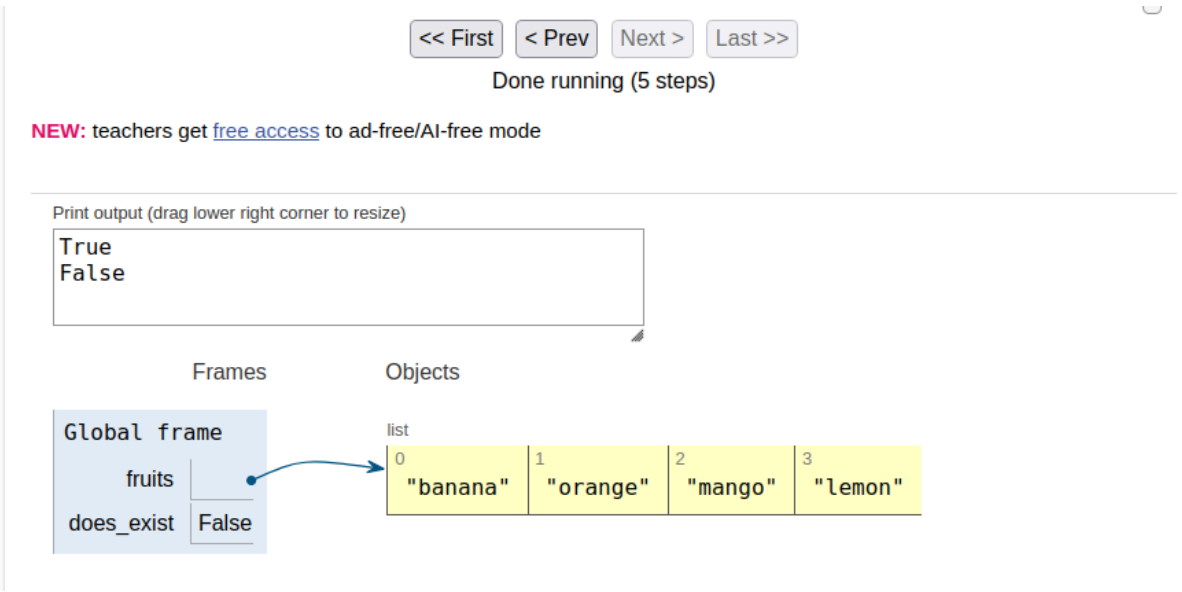
El operador `in` itera el array `ob_item[]` comparando cada puntero con `__eq__` hasta encontrar coincidencia o agotar la lista. Operación O(n).

Comprobar un elemento si es miembro de una lista que se utiliza *en* el operador. Vea el ejemplo a continuación.

Sintaxis:

```
elemento in lista      # retorna True o False
```

python
```
fruits = ['banana', 'orange', 'mango', 'lemon']

does_exist = 'banana' in fruits     # itera ob_item[] buscando "banana" → encontrado en [0]
print(does_exist)                   # True

does_exist = 'lime' in fruits       # itera todos los elementos → no encontrado
print(does_exist)                   # False
```



PythonTutor — Operador `in`

El diagrama muestra `does_exist = False` en el Global Frame (estado final tras la segunda evaluación).

Omisión: `in` no usa LEGB — es un operador del lenguaje compilado a bytecode `CONTAINS_OP`, que llama a `list.__contains__()`. PythonTutor no muestra la iteración interna del array.

1. El "Azúcar Sintáctico"

En Python, el operador in es lo que llamamos **syntactic sugar**. Cuando escribes elemento in lista, Python no hace magia, sino que traduce esa instrucción a una llamada de un método especial (o "magic method").

La jerarquía de ejecución es la siguiente:

1. **Nivel de código:** item in my_list
2. **Nivel de Bytecode:** Se ejecuta la instrucción CONTAINS_OP.
3. **Nivel de Objeto:** Se invoca a my_list.__contains__(item).

2. ¿Cómo funciona list.__contains__?

Como bien mencionaste, la lista en Python (en su implementación estándar CPython) es un array de punteros (ob_item). El método interno hace lo siguiente:

- **Iteración Lineal:** Recorre la lista desde el índice 0 hasta el n-1.
- **Comparación de Igualdad:** En cada paso, comprueba si el objeto actual es igual al que buscas.
- **Cortocircuito:** En cuanto encuentra una coincidencia (True), detiene la ejecución y devuelve True. Si llega al final sin éxito, devuelve False.

3. Diferencia clave: __contains__ vs Iteración manual

Aunque in equivale a un bucle for, es **mucho más rápido** que escribir el bucle tú mismo en Python por una razón sencilla:

> [!IMPORTANT] El método list.__contains__ está escrito en **C**. Esto significa que la iteración ocurre a nivel de lenguaje de máquina, evitando el "overhead" de interpretar cada línea de un bucle for manual en Python.

Añadir artículos a una lista.

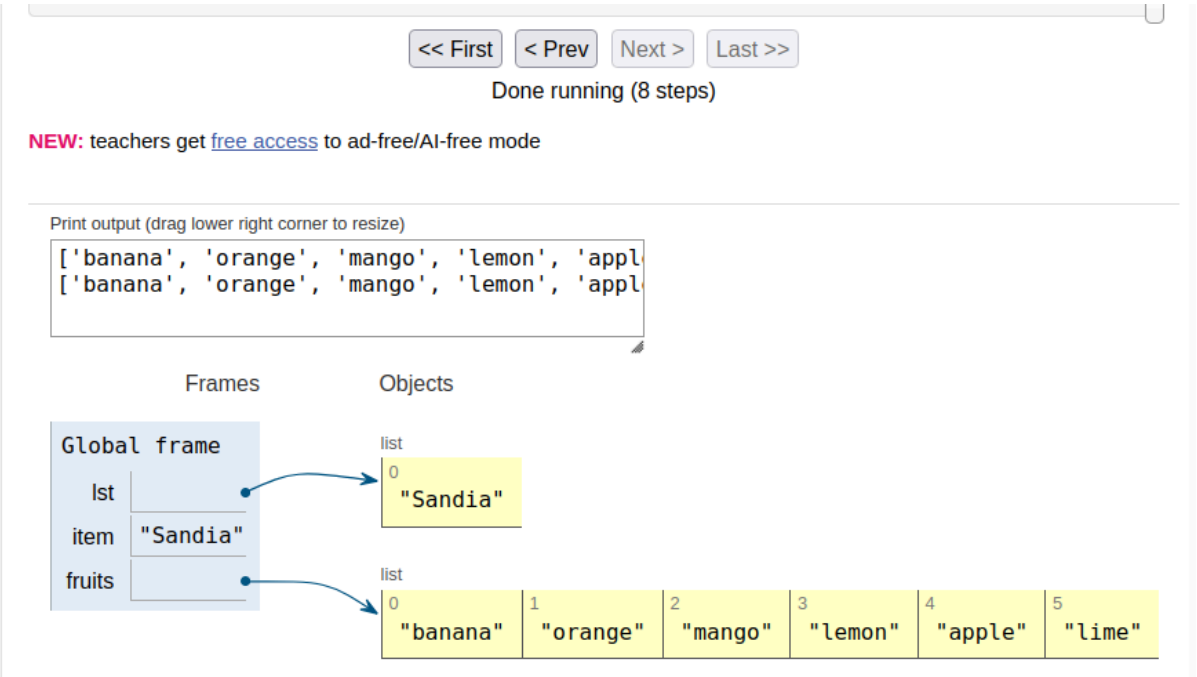
`append()` agrega un elemento al final. Internamente incrementa `ob_size` y escribe el nuevo puntero en `ob_item[ob_size-1]`. Si la capacidad (`ob_alloc`) es insuficiente, CPython realoca el array con más espacio (**sobrealoca** para evitar realocaciones frecuentes).

Sintaxis:

```
# syntax
lst = list()
lst.append(elemento)       # agrega al final, modifica la lista original, retorna None
```

Para añadir un elemento al final de una lista existente utilizamos el método *append*().

```
python
fruits = ['banana', 'orange', 'mango', 'lemon']

fruits.append('apple')      # ob_size: 4→5, ob_item[4] → puntero a "apple"
                            # si ob_alloc era 4, CPython realiza con capacidad mayor
print(fruits)               # ['banana', 'orange', 'mango', 'lemon', 'apple']

fruits.append('lime')       # ob_size: 5→6, ob_item[5] → puntero a "lime"
print(fruits)               # ['banana', 'orange', 'mango', 'lemon', 'apple', 'lime']
```



`append()` muta la lista existente — no crea una nueva. A diferencia del operador `+` entre listas que sí crea un objeto nuevo.

Insertar elementos en una lista

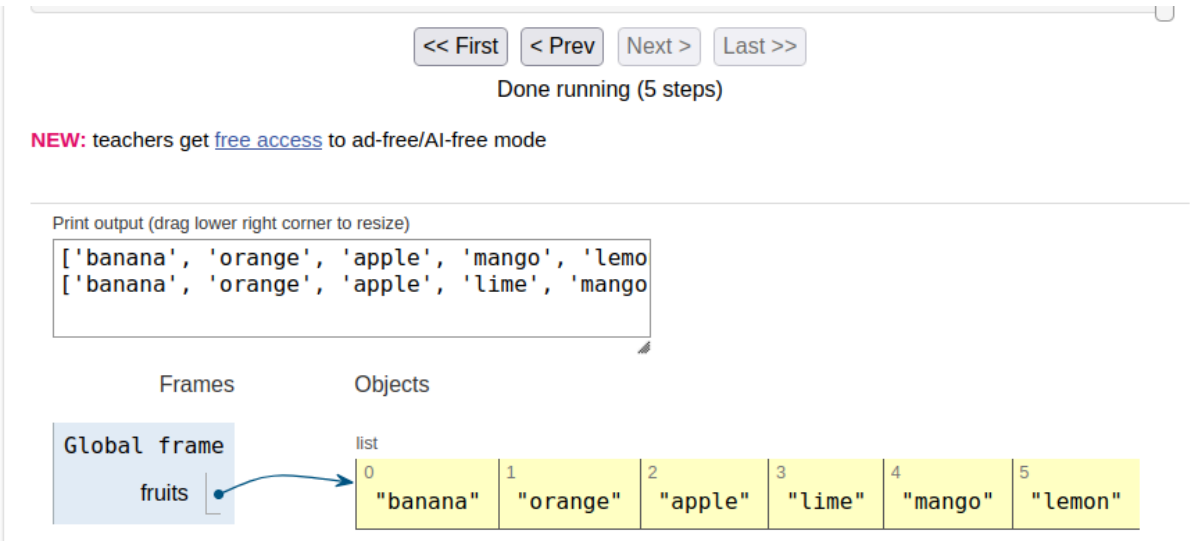
`insert()` coloca un elemento en un índice específico. Todos los elementos desde ese índice en adelante se desplazan una posición a la derecha — operación O(n) porque requiere mover punteros.

Podemos utilizar el método *insert()* para insertar un único elemento en un índice especificado en una lista. Tenga en cuenta que otros elementos se desplazan hacia la derecha. Los métodos *insert()* toman dos argumentos:índice y un elemento para insertar.

```
# syntax
lst = ['item1', 'item2']
lst.insert(indice, elemento)    # insertar elemento en indice, desplaza el resto
```

python

```
fruits = ['banana', 'orange', 'mango', 'lemon']

fruits.insert(2, 'apple')   # desplaza ob_item[2..] → derecha, escribe "apple" en ob_item[2]
print(fruits)               # ['banana', 'orange', 'apple', 'mango', 'lemon']

fruits.insert(3, 'lime')    # inserta "lime" en índice 3, desplaza "mango" y "lemon"
print(fruits)               # ['banana', 'orange', 'apple', 'lime', 'mango', 'lemon']
```



Diferencia con `append()`: `append()` es O(1) amortizado (solo escribe al final). `insert()` es O(n) porque desplaza elementos.

Eliminar artículos de una lista

`remove()` busca la primera ocurrencia del valor especificado, la elimina y desplaza los elementos posteriores a la izquierda. Si el valor no existe lanza `ValueError`. Modifica la lista original.

```
# syntax
lst = ['item1', 'item2']
lst.remove(elemento)    # elimina la PRIMERA ocurrencia, O(n)


fruits = ['banana', 'orange', 'mango', 'lemon', 'banana']

fruits.remove('banana') # busca de izquierda a derecha, elimina índice 0
                         # ob_refcnt de "banana" en [0] decrementa # elementos [1..] se
                        desplazan a la izquierda
print(fruits) # ['orange', 'mango', 'lemon', 'banana'] — el segundo "banana" permanece

fruits.remove('lemon') # elimina índice 2 en la lista actual
print(fruits) # ['orange', 'mango', 'banana']
```

```
1 fruits = ['banana', 'orange', 'mango', 'lemon', 'banana']
2 fruits.remove('banana')
3 print(fruits)  # ['orange', 'mango', 'lemon', 'banana'] - this method rem
4 fruits.remove('lemon')
5 print(fruits)  # ['orange', 'mango', 'banana']
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Done running (5 steps)

NEW: teachers get free access to ad-free/AI-free mode

Print output (drag lower right corner to resize)

```
['orange', 'mango', 'lemon', 'banana']
['orange', 'mango', 'banana']
```

Frames | Objects

Global frame
fruits

list
0 "orange" | 1 "mango" | 2 "banana"

Eliminar artículos usando Pop()

`pop()` elimina y **retorna** el elemento en el índice especificado. Sin argumento, elimina el último elemento. Es O(1) al final, O(n) en posiciones intermedias.

Sintaxis:

```
lst = ['item1', 'item2']
lst.pop()          # elimina y retorna el último elemento
lst.pop(indice)    # elimina y retorna el elemento en indice
```

python

```
fruits = ['banana', 'orange', 'mango', 'lemon']

fruits.pop()          # elimina ob_item[ob_size-1] → "lemon", ob_size: 4→3
print(fruits)         # ['banana', 'orange', 'mango']

fruits.pop(0)         # elimina ob_item[0] → "banana", desplaza resto a la izquierda
print(fruits)         # ['orange', 'mango']
```

```
1 fruits = ['banana', 'orange', 'mango', 'lemon']
2 fruits.pop()
3 print(fruits)        # ['banana', 'orange', 'mango']
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 2 of 3

NEW: teachers get free access to ad-free/AI-free mode

Print output (drag lower right corner to resize)

Frames | Objects

Global frame
fruits

list

0	1	2	3
"banana"	"orange"	"mango"	"lemon"

```
1 fruits = ['banana', 'orange', 'mango', 'lemon']
2 fruits.pop()
3 print(fruits)        # ['banana', 'orange', 'mango']
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Done running (3 steps)

NEW: teachers get free access to ad-free/AI-free mode

Print output (drag lower right corner to resize)

```
['banana', 'orange', 'mango']
```

Frames | Objects

Global frame
fruits

list

0	1	2
"banana"	"orange"	"mango"

PythonTutor — `pop()` (Pasos 2 y 3 de 3)

Paso 2: La lista muestra 4 elementos con `lemon` aún presente. La flecha next line indica que `pop()` está a punto de ejecutarse.

Paso 3: La lista en Objects ahora tiene 3 celdas: `"banana"`, `"orange"`, `"mango"`. `"lemon"` desapareció.

Omisión: `pop()` retorna el objeto eliminado. Si ese retorno no se asigna a ninguna variable, el `ob_refcnt` de `"lemon"` decrementa a 0 y es liberado inmediatamente. La lista sigue apuntando al mismo `PyListObject` — solo cambió `ob_size`.

Diferencia entre `remove()` y `pop()`: `remove()` busca por valor (O(n)) y no retorna nada. `pop()` opera por índice y retorna el elemento eliminado. Al final de la lista, `pop()` es O(1); `remove()` siempre es O(n).

Eliminar artículos usando Del

La instrucción `del` es más versátil: puede eliminar un elemento por índice, un rango mediante slicing, o la variable completa del namespace es decir La palabra clave *del* elimina el índice especificado y también se puede utilizar para eliminar elementos dentro del rango de índice. También puede eliminar la lista por completo

Sintaxis:

python
```
lst = ['item1', 'item2']
del lst[indice]        # elimina un elemento
del lst[start:stop]    # elimina un rango (stop no incluido)
del lst                # elimina la variable del namespace — no solo vacía la lista
```

python
```
fruits = ['banana', 'orange', 'mango', 'lemon', 'kiwi', 'lime']

del fruits[0]          # elimina ob_item[0] ("banana"), desplaza resto
print(fruits)          # ['orange', 'mango', 'lemon', 'kiwi', 'lime']

del fruits[1]          # elimina ob_item[1] ("mango") en la lista actual
print(fruits)          # ['orange', 'lemon', 'kiwi', 'lime']

del fruits[1:3]        # elimina índices 1 y 2 ("lemon","kiwi"), stop=3 no incluido
print(fruits)          # ['orange', 'lime']

del fruits             # elimina la entrada "fruits" del dict globals()
                       # si ob_refcnt del PyListObject llega a 0, se libera el objeto
print(fruits)          # NameError: name 'fruits' is not defined
```

```
1 fruits = ['banana', 'orange', 'mango', 'lemon', 'kiwi', 'lime']
2 del fruits[0]
3 print(fruits)       # ['orange', 'mango', 'lemon', 'kiwi', 'lime']
4 del fruits[1]
5 print(fruits)       # ['orange', 'lemon', 'kiwi', 'lime']
6 del fruits[1:3]     # this deletes items between given indexes, so it doe
7 print(fruits)       # ['orange', 'lime']
8 del fruits
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 2 of 8

NEW: teachers get free access to ad-free/AI-free mode

Print output (drag lower right corner to resize)

Frames | Objects

Global frame

fruits

list

0	1	2	3	4	5
"banana"	"orange"	"mango"	"lemon"	"kiwi"	"lime"

```
1 fruits = ['banana', 'orange', 'mango', 'lemon', 'kiwi', 'lime']
2 del fruits[0]
3 print(fruits)       # ['orange', 'mango', 'lemon', 'kiwi', 'lime']
4 del fruits[1]
5 print(fruits)       # ['orange', 'lemon', 'kiwi', 'lime']
6 del fruits[1:3]     # this deletes items between given indexes, so it doe
7 print(fruits)       # ['orange', 'lime']
8 del fruits
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 3 of 8

NEW: teachers get free access to ad-free/AI-free mode

Print output (drag lower right corner to resize)

Frames | Objects

Global frame

fruits

list

0	1	2	3	4
"orange"	"mango"	"lemon"	"kiwi"	"lime"

```
1 fruits = ['banana', 'orange', 'mango', 'lemon', 'kiwi', 'lime']
2 del fruits[0]
3 print(fruits)       # ['orange', 'mango', 'lemon', 'kiwi', 'lime']
4 del fruits[1]
5 print(fruits)       # ['orange', 'lemon', 'kiwi', 'lime']
6 del fruits[1:3]     # this deletes items between given indexes, so it doe
7 print(fruits)       # ['orange', 'lime']
8 del fruits
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 5 of 8

NEW: teachers get free access to ad-free/AI-free mode

Print output (drag lower right corner to resize)

```
['orange', 'mango', 'lemon', 'kiwi', 'lime']
```

Frames | Objects

Global frame

fruits

list

0	1	2	3
"orange"	"lemon"	"kiwi"	"lime"

```
1 fruits = ['banana', 'orange', 'mango', 'lemon', 'kiwi', 'lime']
2 del fruits[0]
3 print(fruits)       # ['orange', 'mango', 'lemon', 'kiwi', 'lime']
4 del fruits[1]
5 print(fruits)       # ['orange', 'lemon', 'kiwi', 'lime']
6 del fruits[1:3]     # this deletes items between given indexes, so it doe
7 print(fruits)       # ['orange', 'lime']
8 del fruits
```

Edit this code

line that just executed
next line to execute

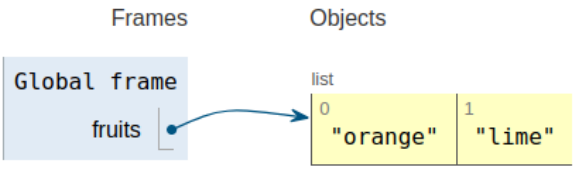
<< First | < Prev | Next > | Last >>

Step 7 of 8

NEW: teachers get free access to ad-free/AI-free mode

Print output (drag lower right corner to resize)

```
['orange', 'mango', 'lemon', 'kiwi', 'lime']
['orange', 'lemon', 'kiwi', 'lime']
```



```
1 fruits = ['banana', 'orange', 'mango', 'lemon', 'kiwi', 'lime']
2 del fruits[0]
3 print(fruits)       # ['orange', 'mango', 'lemon', 'kiwi', 'lime']
4 del fruits[1]
5 print(fruits)       # ['orange', 'lemon', 'kiwi', 'lime']
6 del fruits[1:3]     # this deletes items between given indexes, so it doe
7 print(fruits)       # ['orange', 'lime']
8 del fruits
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 8 of 8

NEW: teachers get free access to ad-free/AI-free mode

Print output (drag lower right corner to resize)

```
['orange', 'mango', 'lemon', 'kiwi', 'lime']
['orange', 'lemon', 'kiwi', 'lime']
['orange', 'lime']
```

Frames | Objects

Global frame
fruits

list
0 "orange" | 1 "lime"

PythonTutor — `del` (Pasos 2 a 8)

Paso 2 de 8: La lista muestra 6 elementos completos. La flecha indica que `del fruits[0]` está a punto de ejecutarse.

Paso 3 de 8: La lista ahora tiene 5 elementos empezando con `"orange"` en índice 0. El array fue desplazado hacia la izquierda.

Paso 5 de 8: Tras `del fruits[1]`, la lista tiene 4 elementos: `"orange"`, `"lemon"`, `"kiwi"`, `"lime"`.

Paso 7 de 8: Tras `del fruits[1:3]`, la lista tiene solo 2 elementos: `"orange"`, `"lime"`. Los índices 1 y 2 fueron eliminados.

Paso 8 de 8 — `del fruits`: La entrada `fruits` desaparece del Global Frame. **Omisión crítica y diferencia con `clear()`:** `del lst` elimina el nombre del namespace — si nadie más referencia el `PyListObject`, éste se libera. `lst.clear()` vacía el contenido pero el

objeto lista permanece en el heap y la variable sigue válida en el namespace apuntando a una lista vacía.

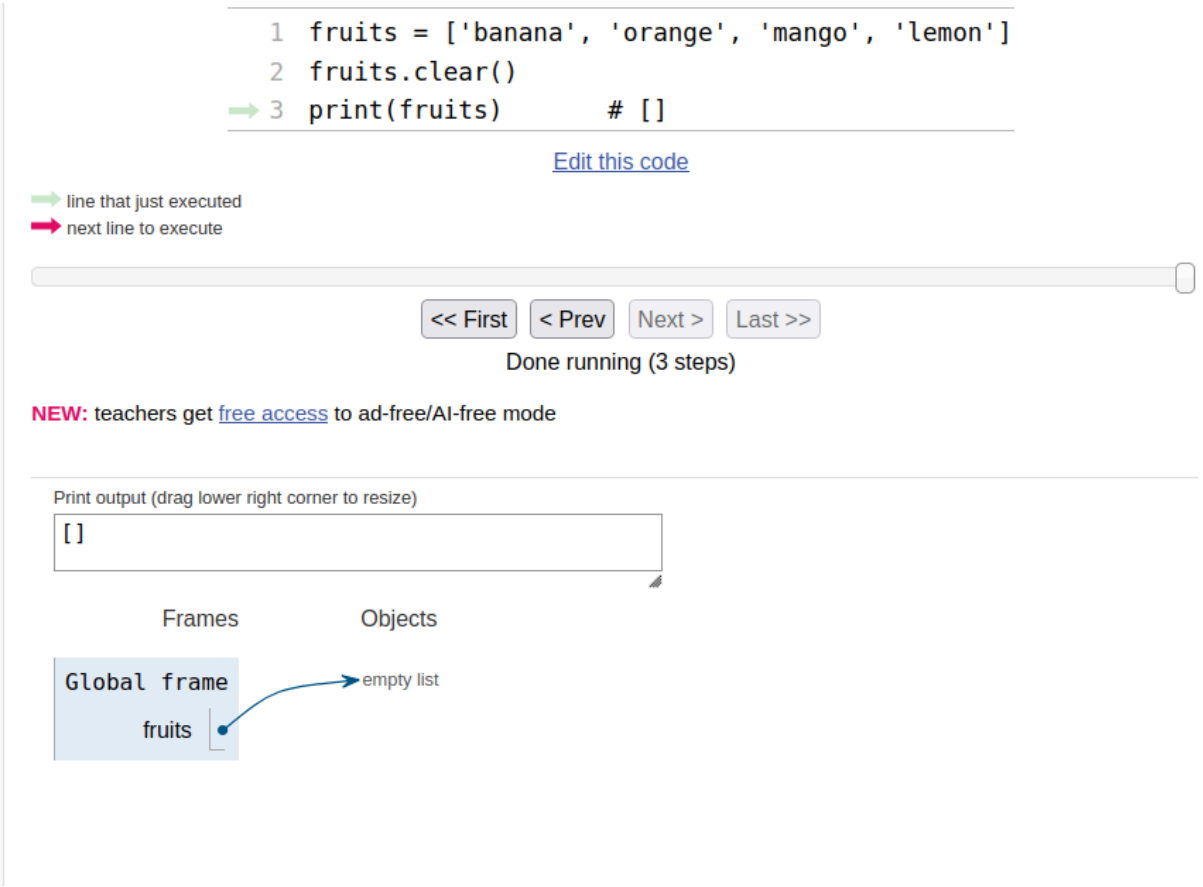
Lista de compensación Artículos

`clear()` elimina todos los elementos internos pero conserva el objeto lista en el heap y la variable en el namespace.

Sintaxis:

python
```
lst = ['item1', 'item2']
lst.clear()    # vacía el contenido, ob_size → 0, la variable sigue válida
```

python
```
fruits = ['banana', 'orange', 'mango', 'lemon']

fruits.clear()    # libera los punteros internos, ob_size=0, ob_alloc puede mantenerse
print(fruits)     # []  ← la variable sigue existiendo, apunta a lista vacía
```



PythonTutor — `clear()`

El diagrama final muestra `fruits` en el Global Frame apuntando a "empty list" en Objects. **Diferencia clave con `del fruits`:** la variable `fruits` sigue existiendo y sigue apuntando a un `PyListObject` válido — simplemente está vacío. Con `del fruits` la variable desaparece del namespace.

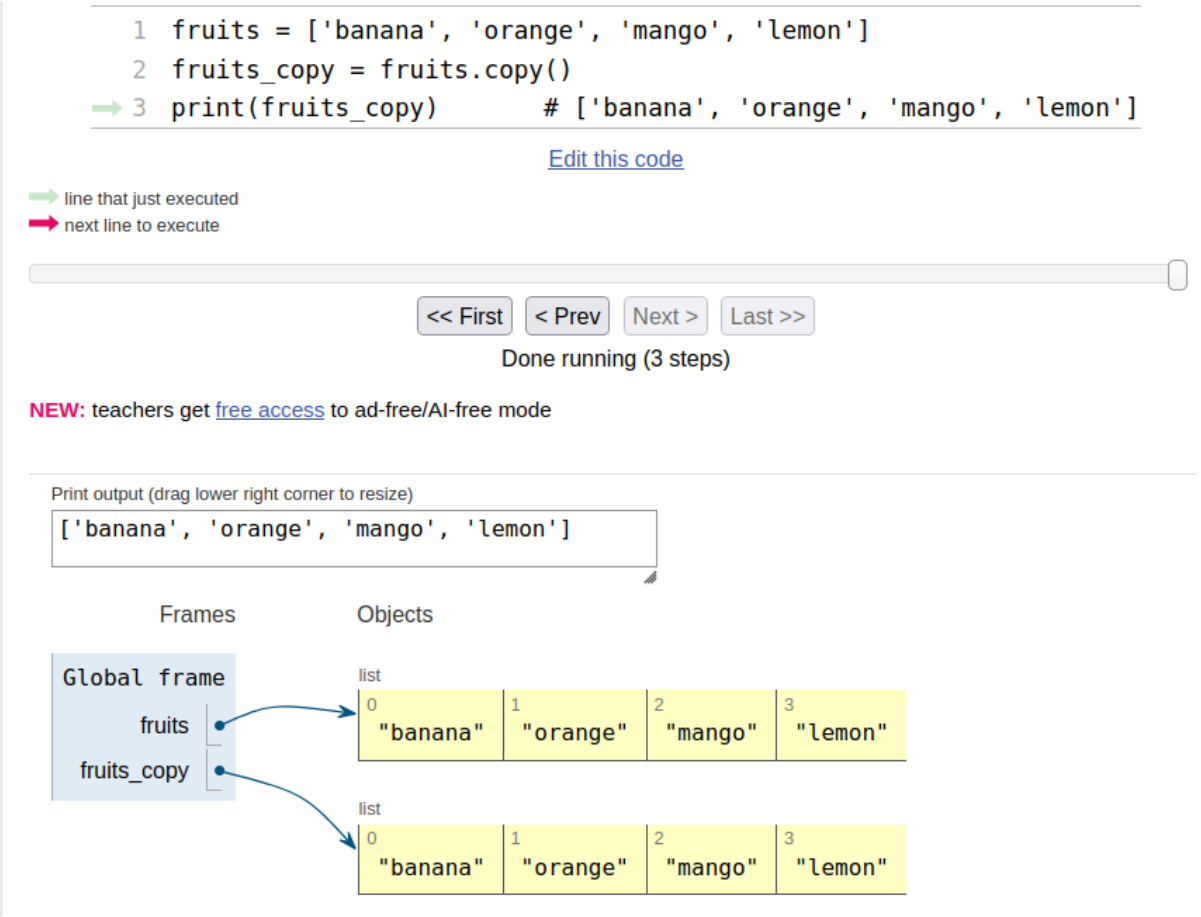
Copiar una lista

Asignar una lista a otra variable (`list2 = list1`) no crea una copia — ambas variables apuntan al mismo objeto en el heap. `copy()` crea un nuevo `PyListObject` con los mismos punteros copiados (shallow copy).

Sintaxis:

```python
lst_copy = lst.copy()      # shallow copy: nuevo PyListObject, mismos punteros internos
lst_copy = lst[:]          # equivalente mediante slicing
```

```python
fruits = ['banana', 'orange', 'mango', 'lemon']

fruits_copy = fruits.copy()     # crea nuevo PyListObject en heap
                                # ob_item[] del nuevo objeto contiene los mismos punteros
print(fruits_copy)              # ['banana', 'orange', 'mango', 'lemon']
```



PythonTutor — `copy()`

El diagrama muestra `fruits` y `fruits_copy` como dos entradas distintas en el Global Frame, cada una apuntando a una lista separada en Objects. Ambas listas tienen el mismo contenido.

Omisión clave: los strings dentro de ambas listas son los mismos objetos — `fruits[0]` y `fruits_copy[0]` apuntan al mismo `PyUnicodeObject "banana"`.

Modificar `fruits_copy[0] = 'kiwi'` no afecta `fruits`, pero si el elemento fuera una lista mutable, modificar su contenido sí afectaría ambas (por eso se llama shallow copy).

El caso del "Kiwi" (Reemplazo)

Cuando haces fruits_copy[0] = 'kiwi', estás rompiendo el vínculo.

- **Antes:** El índice 0 de ambas listas apuntaba al objeto "banana".
- **Después:** El índice 0 de la lista copia ahora apunta a un objeto nuevo llamado "kiwi".
- **Resultado:** La lista original no se entera, porque ella sigue apuntando a "banana". Simplemente cambiaste un puntero por otro.

La confusión viene de mezclar **dos niveles distintos de “mutabilidad”**:

1. **La lista como contenedor (mutable)**
2. **Los objetos dentro de la lista (que pueden ser mutables o no)**

🔹 1. La lista *sí* es mutable

Correcto: una lista en Python es mutable. Eso significa que podés hacer cosas como:

```
fruits = ['banana', 'orange']
fruits[0] = 'kiwi'
```

Ahí estás **modificando la lista**, porque cambiás qué objeto está en la posición 0.

🔹 2. Pero los elementos pueden ser mutables o inmutables

- Strings ("banana") → **inmutables**
- Listas ([1, 2]) → **mutables**

🔹 3. Qué pasa con copy() (shallow copy)

Cuando hacés:

```
fruits = ['banana', 'orange']
fruits_copy = fruits.copy()
```

Tenés:

- Dos listas distintas (dos contenedores)
- Pero **los mismos objetos adentro**

Visualmente:

```
fruits ------> ['banana', 'orange']
                ↑        ↑
fruits_copy -> ['banana', 'orange']
```

Ambas listas apuntan a los mismos strings.

🔸 Caso 1: Reemplazar un elemento (NO afecta a la original)

```
fruits_copy[0] = 'kiwi'
```

Esto:

- NO modifica el string "banana" (ni se puede, es inmutable)
- Solo cambia el puntero dentro de fruits_copy

Resultado:

```
fruits      = ['banana', 'orange']
fruits_copy  = ['kiwi', 'orange']
```

✔ No afecta a fruits

🔸 Caso 2: Elemento mutable (SÍ afecta a ambas)

```
fruits = [['banana'], ['orange']]
fruits_copy = fruits.copy()

fruits_copy[0].append('kiwi')
```

Acá:

- fruits[0] y fruits_copy[0] apuntan a **la misma lista interna**
- .append() modifica esa lista interna

Resultado:

```
fruits      = [['banana', 'kiwi'], ['orange']]
fruits_copy  = [['banana', 'kiwi'], ['orange']]
```

✔ Ambas cambian

🔑 La clave (lo importante de verdad)

- fruits_copy[0] = 'kiwi' → cambia **referencia** (puntero) → no afecta a la original
- fruits_copy[0].append(...) → modifica **el objeto compartido** → sí afecta

```
        fruits          fruits_copy
          |                 |
          ▼                 ▼
PyListObject A    PyListObject B  ← objetos distintos
[ptr0|ptr1|...]   [ptr0|ptr1|...]
   |                  |
   └──────────────────┘
            |
      mismos objetos string en heap
```

Listas de unión

Operador +

Crea un tercer `PyListObject` nuevo copiando todos los punteros de ambas listas. No modifica ninguna lista original.

Sintaxis:

python
```
lista3 = lista1 + lista2    # nuevo objeto, O(n+m)
```
python

```
positive_numbers = [1, 2, 3, 4, 5]
zero = [0]
negative_numbers = [-5, -4, -3, -2, -1]

integers = negative_numbers + zero + positive_numbers
# evalúa de izquierda a derecha:
# 1) negative_numbers + zero → nuevo objeto temporal [−5,−4,−3,−2,−1,0]
# 2) temporal + positive_numbers → nuevo objeto integers [−5,...,5]
# el objeto temporal queda sin referencias → ob_refcnt=0 → se libera
print(integers)            # [-5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5]

fruits = ['banana', 'orange', 'mango', 'lemon']
vegetables = ['Tomato', 'Potato', 'Cabbage', 'Onion', 'Carrot']
fruits_and_vegetables = fruits + vegetables     # nuevo objeto, fruits y vegetables no
cambian
print(fruits_and_vegetables)
# ['banana', 'orange', 'mango', 'lemon', 'Tomato', 'Potato', 'Cabbage', 'Onion', 'Carrot
```

Python 3.11
known limitations

```
1 positive_numbers = [1, 2, 3, 4, 5]
2 zero = [0]
3 negative_numbers = [-5,-4,-3,-2,-1]
4 integers = negative_numbers + zero + positive_numbers
5 print(integers) # [-5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5]
```

Edit this code

line that just executed
next line to execute

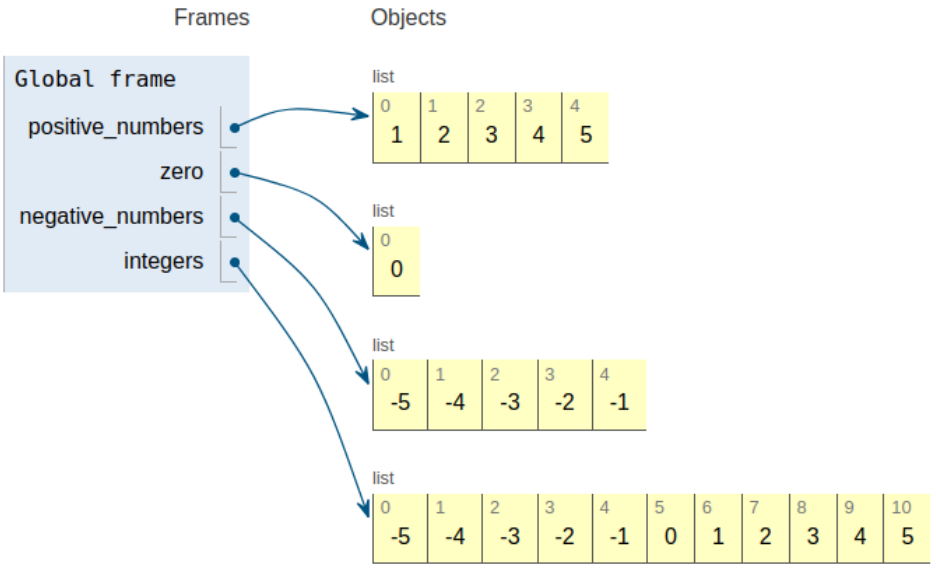
<< First | < Prev | Next > | Last >>

Done running (5 steps)

NEW: teachers get free access to ad-free/AI-free mode

Print output (drag lower right corner to resize)

```
[-5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5]
```



Método `extend()`

`extend()` agrega los elementos de una segunda lista al final de la primera, mutando el objeto original. No crea un objeto lista nuevo. Es más eficiente que `+` cuando se quiere modificar una lista existente.

Unir usando el método extend() El método *extend()* permite añadir la lista en una lista. Vea el ejemplo a continuación.

```
# syntax
list1 = ['item1', 'item2']
list2 = ['item3', 'item4', 'item5']
```

```
list1.extend(list2) # ['item1', 'item2', 'item3', 'item4', 'item5']
                    # modifica lista1 en su lugar, retorna None
```

python

```
num1 = [0, 1, 2, 3]
num2 = [4, 5, 6]

num1.extend(num2)      # copia los punteros de num2 al final de num1, ob_size de num1: 4→7
                    # num2 NO se modifica
print('Numbers:', num1) # Numbers: [0, 1, 2, 3, 4, 5, 6]
```

PythonTutor — `extend()` con números

El diagrama muestra `num1` apuntando a la lista resultado de 7 elementos, y `num2` apuntando a su lista original de 3 elementos sin modificar.

Diferencia clave con `+`: `extend()` muta `num1` (mismo `PyListObject`, nueva capacidad si necesario). `+` crearía un tercer objeto dejando ambos originales intactos.

python

```
negative_numbers = [-5, -4, -3, -2, -1]
positive_numbers = [1, 2, 3, 4, 5]
zero = [0]

negative_numbers.extend(zero)            # negative_numbers ahora tiene 6 elementos
negative_numbers.extend(positive_numbers)  # negative_numbers ahora tiene 11 elementos
print('Integers:', negative_numbers)
# Integers: [-5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5]

fruits = ['banana', 'orange', 'mango', 'lemon']
vegetables = ['Tomato', 'Potato', 'Cabbage', 'Onion', 'Carrot']
fruits.extend(vegetables)      # muta fruits, agrega 5 punteros más
print('Fruits and vegetables:', fruits)
# Fruits and vegetables: ['banana', 'orange', 'mango', 'lemon', 'Tomato', 'Potato', 'Cabbage', 'Onion', 'Carrot']
```

Python 3.11
known limitations

```
1 num1 = [0, 1, 2, 3]
2 num2= [4, 5, 6]
3 num1.extend(num2)
4 print('Numbers:', num1) # Numbers: [0, 1, 2, 3, 4, 5, 6]
```

Edit this code

line that just executed
next line to execute

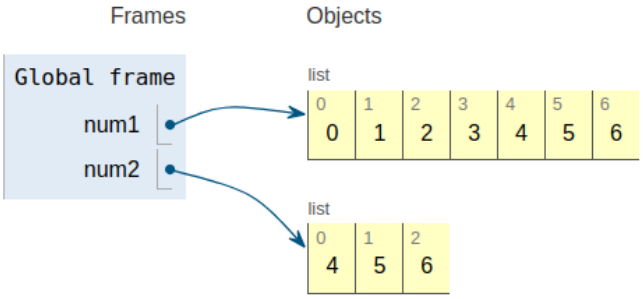
<< First | < Prev | Next > | Last >>

Done running (4 steps)

NEW: teachers get free access to ad-free/AI-free mode

Print output (drag lower right corner to resize)

```
Numbers: [0, 1, 2, 3, 4, 5, 6]
```



Contar artículos en una lista

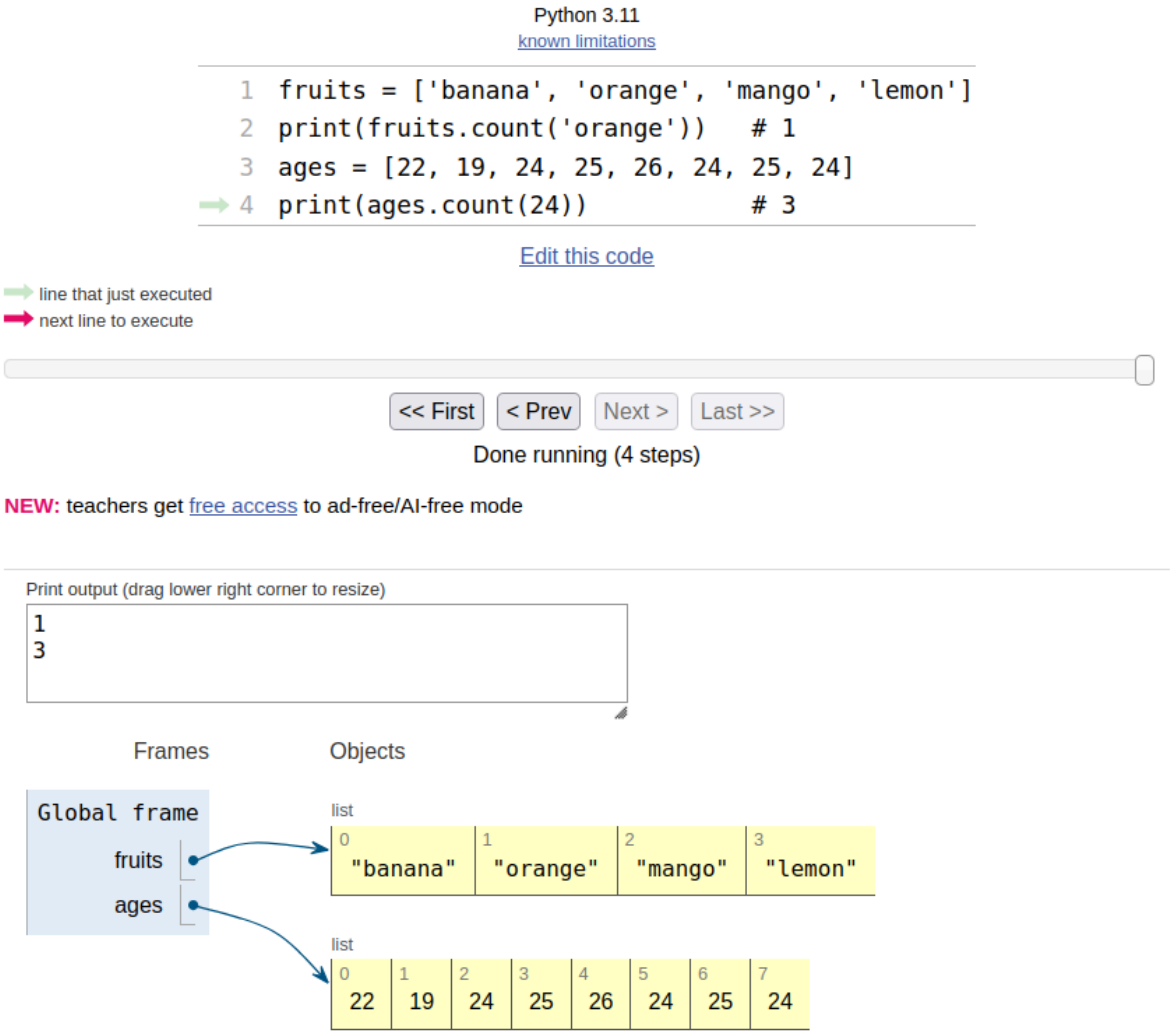
El método *count()* devuelve el número de veces que un elemento aparece en una lista:

`count()` itera el array completo contando cuántas veces aparece el valor. Operación O(n).

Sintaxis:

python
lst.count(elemento) # retorna int con la cantidad de ocurrencias
python
fruits = ['banana', 'orange', 'mango', 'lemon']
print(fruits.count('orange')) # itera 4 elementos, encuentra 1 coincidencia → 1

ages = [22, 19, 24, 25, 26, 24, 25, 24]
print(ages.count(24)) # itera 8 elementos, encuentra 3 coincidencias → 3



Buscar índice — `index()`

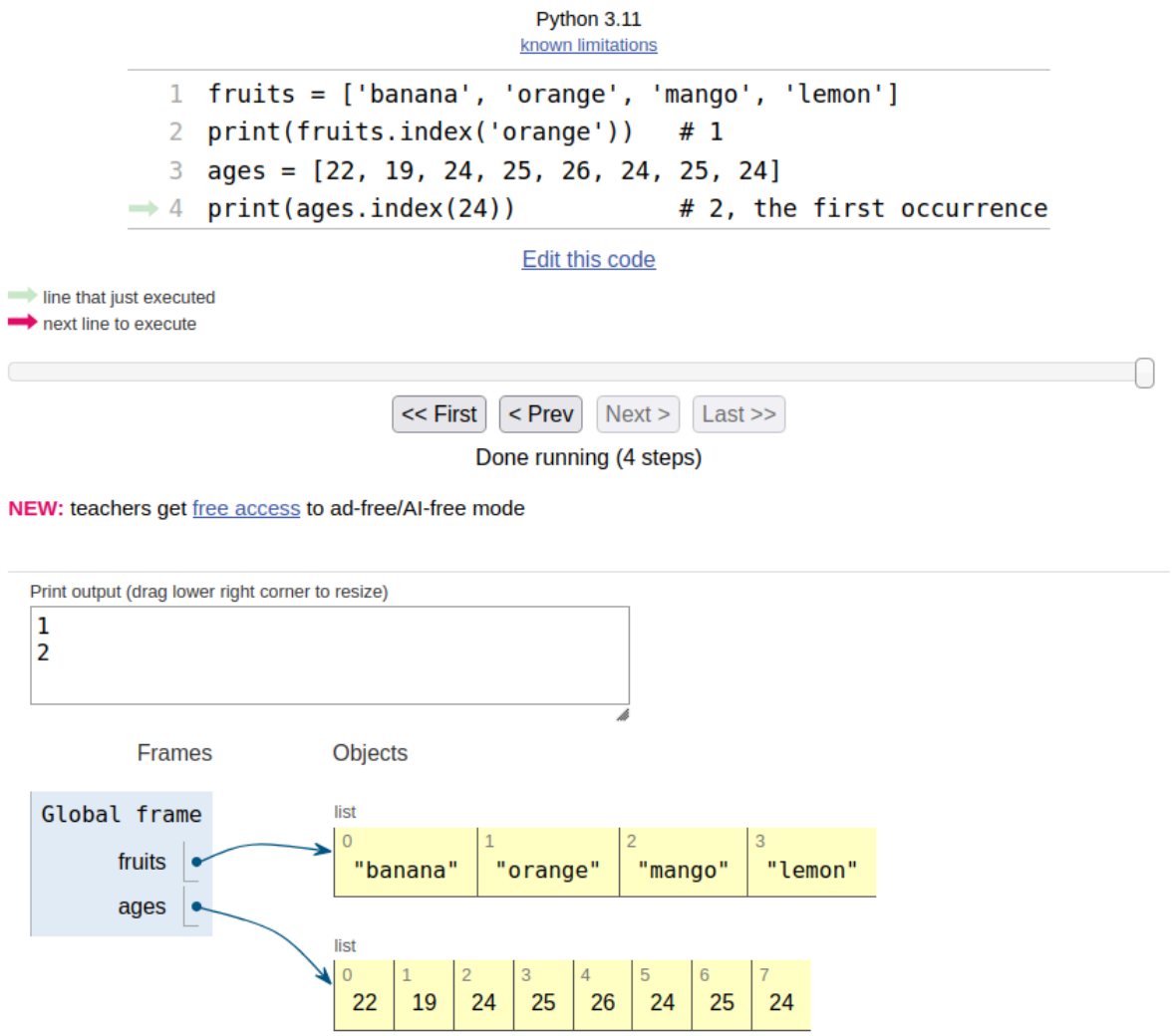
`index()` retorna el índice de la primera ocurrencia del valor. Lanza `ValueError` si no existe. Operación O(n).

Sintaxis:

python
lst.index(elemento) # retorna el índice de la primera ocurrencia
python
fruits = ['banana', 'orange', 'mango', 'lemon']
print(fruits.index('orange')) # itera hasta encontrar "orange" en posición 1 → 1

ages = [22, 19, 24, 25, 26, 24, 25, 24]
print(ages.index(24)) # primera ocurrencia de 24 está en índice 2 → 2

Diferencia con `count()`: `index()` detiene la búsqueda al encontrar la primera coincidencia. `count()` siempre recorre toda la lista.



Revertir una lista — `reverse()`

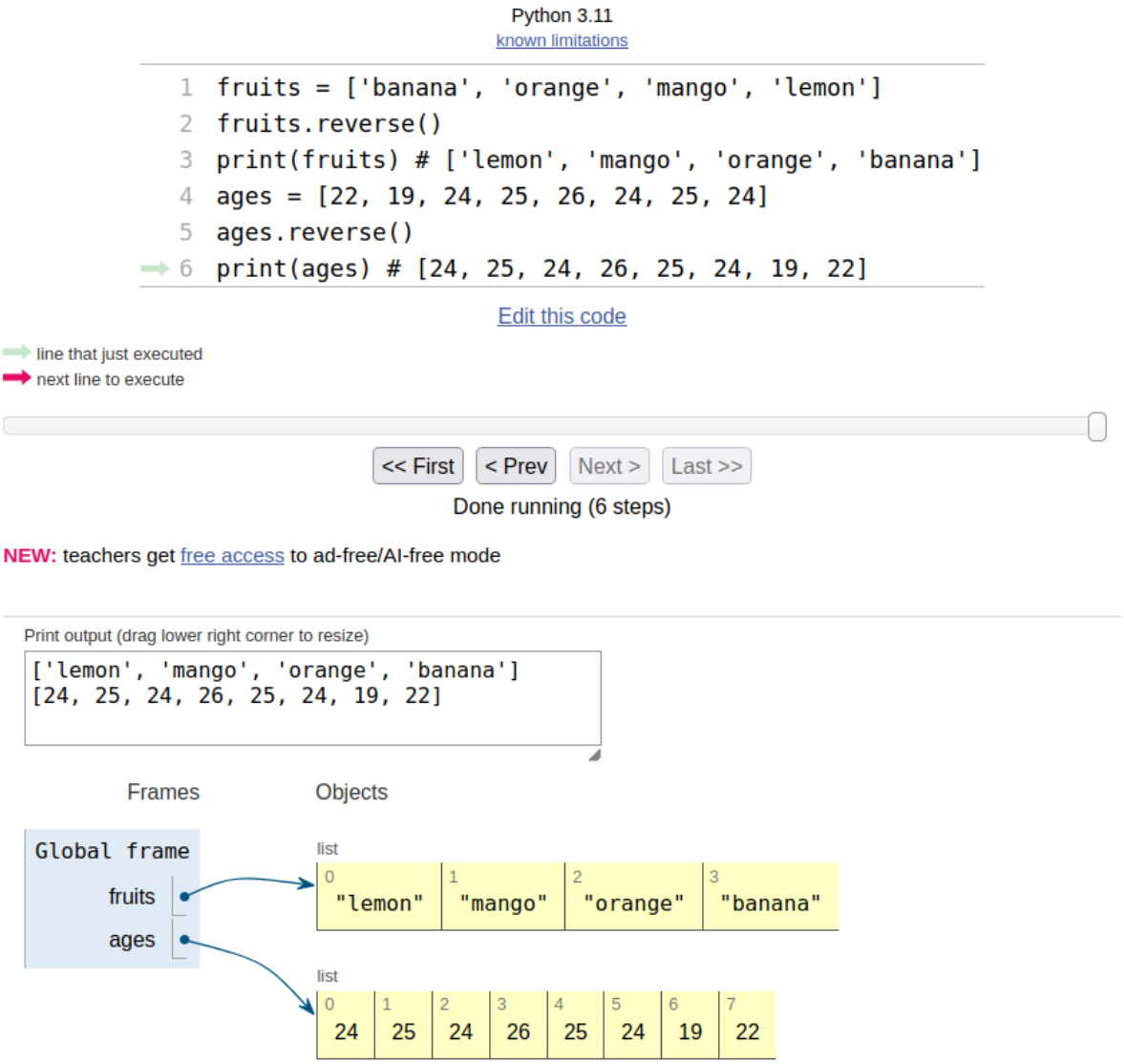
`reverse()` invierte el orden del array `ob_item[]` **in place**, intercambiando punteros desde los extremos hacia el centro. No crea un nuevo objeto lista.

Sintaxis:

python
```
lst.reverse()      # muta la lista original, retorna None
```
python
```
fruits = ['banana', 'orange', 'mango', 'lemon']
fruits.reverse()      # intercambia ob_item[0]↔ob_item[3], ob_item[1]↔ob_item[2]
print(fruits)         # ['lemon', 'mango', 'orange', 'banana']
```

```
ages = [22, 19, 24, 25, 26, 24, 25, 24]
ages.reverse()
print(ages)          # [24, 25, 24, 26, 25, 24, 19, 22]
```

Diferencia con `[::-1]`: `reverse()` muta la lista original O(n/2). `fruits[::-1]` crea un nuevo `PyListObject` con los punteros en orden inverso — la lista original no cambia.



Lista de clasificación de artículos

Para ordenar listas podemos utilizar el método *sort()* o las funciones integradas *sorted()*. El método *sort()* reordena los elementos de la lista en orden ascendente y modifica la lista original. Si un argumento del método *sort()* inverso es igual a true, organizará la lista en orden descendente.

Ordenar una lista — `sort()` y `sorted()`

`sort()` — ordena in place

Reordena el array `ob_item[]` usando el algoritmo Timsort (O(n log n)). Modifica el objeto lista existente, retorna `None`.

Sintaxis:

python
```
lst.sort()              # ascendente
lst.sort(reverse=True)     # descendente
```
python
```
fruits = ['banana', 'orange', 'mango', 'lemon']
fruits.sort()            # Timsort sobre ob_item[], reordena punteros alfabéticamente
print(fruits)            # ['banana', 'lemon', 'mango', 'orange']

fruits.sort(reverse=True)   # ordena descendente
print(fruits)            # ['orange', 'mango', 'lemon', 'banana']

ages = [22, 19, 24, 25, 26, 24, 25, 24]
ages.sort()
print(ages)              # [19, 22, 24, 24, 24, 25, 25, 26]

ages.sort(reverse=True)
print(ages)              # [26, 25, 25, 24, 24, 24, 22, 19]
```

PythonTutor — `sort()` (estado final)

El diagrama muestra `fruits` apuntando a la misma lista (mismo objeto) pero con los elementos reordenados: `"orange"`, `"mango"`, `"lemon"`, `"banana"` (resultado tras `sort(reverse=True)`). **Omisión:**

Timsort usa un array auxiliar temporal durante la operación — no visible en el diagrama.

```
fruits = ['banana', 'orange', 'mango', 'lemon']
fruits.sort()
print(fruits)              # sorted in alphabetical order, ['banana', 'le
fruits.sort(reverse=True)
print(fruits) # ['orange', 'mango', 'lemon', 'banana']
ages = [22, 19, 24, 25, 26, 24, 25, 24]
ages.sort()
print(ages) #  [19, 22, 24, 24, 24, 25, 25, 26]

ages.sort(reverse=True)
```

Edit this code

line that just executed
next line to execute

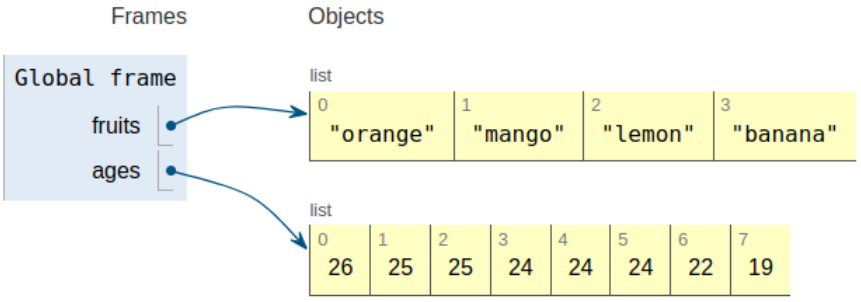
<< First | < Prev | Next > | Last >>

Done running (10 steps)

NEW: teachers get free access to ad-free/AI-free mode

Print output (drag lower right corner to resize)

```
['banana', 'lemon', 'mango', 'orange']
['orange', 'mango', 'lemon', 'banana']
[19, 22, 24, 24, 24, 25, 25, 26]
[26, 25, 25, 24, 24, 24, 22, 19]
```



`sorted()` — retorna nueva lista ordenada

`sorted()` es una función builtin (no un método) que retorna un **nuevo** `PyListObject` ordenado, sin modificar el original.

Sintaxis:

python
nueva = sorted(lst) # nueva lista ordenada, original intacta
nueva = sorted(lst, reverse=True) # nueva lista ordenada descendente
python
fruits = ['banana', 'orange', 'mango', 'lemon']
print(sorted(fruits)) # ['banana', 'lemon', 'mango', 'orange'] — nuevo objeto

```
# fruits sigue siendo ['banana', 'orange', 'mango', 'lemon'] sin cambios

fruits = sorted(fruits, reverse=True)
# ahora fruits apunta al NUEVO objeto ordenado descendente
# el objeto original queda sin referencias → ob_refcnt=0 → se libera
print(fruits)                    # ['orange', 'mango', 'lemon', 'banana']
```

Python 3.11
known limitations

```
1 fruits = ['banana', 'orange', 'mango', 'lemon']
2 print(sorted(fruits))   # ['banana', 'lemon', 'mango', 'orange']
3 # Reverse order
4 fruits = ['banana', 'orange', 'mango', 'lemon']
5 fruits = sorted(fruits,reverse=True)
```

Edit this code

line that just executed
next line to execute

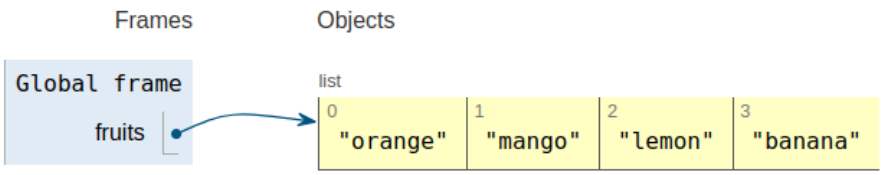
<< First | < Prev | Next > | Last >>

Done running (4 steps)

NEW: teachers get free access to ad-free/AI-free mode

Print output (drag lower right corner to resize)

```
['banana', 'lemon', 'mango', 'orange']
```



Diferencia entre `sort()` y `sorted()`:

	`sort()`	`sorted()`
Tipo	Método de lista	Función builtin
Modifica original	Sí	No
Retorna	`None`	Nueva lista
Uso	Cuando no necesitás el original	Cuando necesitás ambas versiones